

MEMORIA FINAL DEL PROYECTO DE CONTROL DE CALIDAD, EMPAQUETADO Y PALETIZADO DE AZULEJOS



Finn Alair Perea Oltmann

faperolt@upv.edu.es

Claudia Moreno Martínez

cmormar9@upv.edu.es

Ainhoa López Gómez

alopgom3@upv.edu.es

Paula Barona Terol

pbarter@upv.edu.es

ÍNDICE

1. Resumen	3
2. Introducción	4
3. Propuesta de Automatización	5
3.1. Análisis de Alternativas	5
3.2. Objetivos del Proyecto	5
3.3. Otros Aspectos	8
3.4. Impactos adicionales	9
3.5. Límites y Restricciones del Proyecto	9
4. Diseño de la Solución	11
4.1. Elementos y Distribución del Espacio de Trabajo	11
4.2. Estrategias y Algoritmos de Programación Avanzada	21
4.2.1. Algoritmo de Hashing y Búfer Circular	21
4.2.2. Estructuras de Datos Compuestas	21
4.3. Diseño de la Base de Datos	22
4.4. Arquitectura y Diseño de una solución de Integración	22
5. Desarrollo e Implementación	24
5.1. Gestión del Proyecto	24
5.1.1 Plan detallado e implementación del proyecto de asignatura	24
5.1.2 Listado de entregables del proyecto	29
5.2. Pruebas y Validación	29
5.2.1 Descripción de la simulación del proyecto de automatización (RoboDK)	30
5.2.2 Procedimientos de validación y resultados esperados	31
5.3. Costes y Beneficios	31
5.3.1 Presupuesto detallado	31
5.3.2 Análisis Coste-Beneficio	34
5.3.3 ROI	36
6. Normativa y regulación. Seguridad	37
6.1. Impacto en los puestos de trabajo	37
6.2. Consideraciones de seguridad	37
6.3. Cumplimiento de regulaciones y estándares	37
6.4. Documentación y material entregable al cliente	38
7. Desarrollo de Software y Algoritmos	39
7.1. Listado tecnológico	39
7.1.1 Componentes del Sistema (Hardware e Infraestructura Virtual)	39
7.1.2. Tecnologías de Implementación (Software y Entornos de Desarrollo)	41
7.2. Descripción de la implementación	42
7.2.1 Estructura de los programas, división en componentes, tareas y funciones principales	42
7.2.2 Algoritmos de programación avanzada y estructuras de datos de programación	47



7.3. Esquema de la BD utilizada	49
7.4. Repositorio de código en GitHub	50
8. Conclusiones y Recomendaciones	50
9. Referencias	52
10. Anexos	53
10.1. Manual de Instalación y Funcionamiento	53
10.2. Manual del Programador	55
11. Bibliografía	57

1. Resumen

El proyecto desarrollado aborda el diseño e implantación de una celda de automatización para la línea de trabajo enfocada en la inspección, empaquetado y paletizado en el sector cerámico. La solución técnica propuesta permite transformar un proceso tradicionalmente manual, cuya tasa de error es elevada, a un gemelo digital que cubre todas las necesidades. Para ello, se han integrado robots colaborativos controlados por la API Python, implementación de microcontrolador ESP32-S3, mensajería asíncrona MQTT y una base de datos.

Con la implementación se ha buscado desarrollar un sistema flexible y continuo que garantice la sincronización en tiempo real. De esta forma, se obtiene la eliminación total de riesgos laborales para los operarios y la optimización de la eficiencia global de la planta de trabajo mediante la eliminación de cuellos de botella. Sin embargo, para lograr estos beneficios se debe resolver previamente el control de las coordenadas cinemáticas en el simulador RoboDK para prevenir posibles colisiones de la unidad robótica con el entorno. Así pues, de igual manera se debe solventar todo conflicto generado por las comunicaciones inalámbricas que se llevan a cabo a través del broker MQTT.

El desarrollo y ejecución de este proyecto ha sido posible gracias a la Universidad Politécnica de Valencia (UPV) y, en especial a la Escuela Técnica Superior de Ingeniería Informática (ETSINF) por dotarnos de un entorno académico, laboratorios y licencias de simulación profesional para la formación académica y especializada en los ámbitos de robótica e informática industrial.

Finalmente, a todos los docentes de las asignaturas del grado de Informática Industrial y Robótica por su implicación en la docencia; además de impulsar iniciativas como la Feria de Proyectos de Robótica y Ciencia de Datos que nos han permitido ampliar los conocimientos teóricos adquiridos en los seminarios en relación con la gestión de proyectos y automatizado.

2. Introducción

El proyecto de automatización que se ha llevado a cabo se enmarca dentro del sector cerámico. Este sector se trata de una industria de alto rendimiento que exige niveles críticos de precisión, eficiencia y control de calidad en sus líneas de producción.

La industria española de cerámica representa el 22.2% del PIB industrial de la Comunidad Valenciana y supone más del 32.2% del total del PIB de la provincia de Castellón. En este contexto, el control de calidad, empaquetado y paletizado de azulejos constituye una de las áreas con mayor impacto tanto en la productividad como en la calidad final del producto.

Tradicionalmente, las plantas de fabricación operan bajo flujos masivos de materiales de manera continua. El proceso productivo de la empresa cerámica abarca desde la preparación de materias primas, prensado y cocción en los hornos industriales, hasta llegar al final de la línea de producción. En esta última etapa del proceso es donde los productos terminados deben ser inspeccionados para clasificar su calidad, separar los productos conformes de los defectuosos, en buen estado o rotos, y agruparlas a través de cajas en palets para su posterior retirada y puesta en venta. El alto flujo de trabajo obliga a que los procesos de clasificación, empaquetado y paletizado se desarrollen de manera continua, asíncrona y con el menor porcentaje de errores, con la finalidad de minimizar los cuellos de botella.

Para abordar la automatización de la línea de producción de productos cerámicos, el grupo de trabajo ha asumido un rol de consultora externa. Se ha optado por este rol, ya que de esta forma se pueden aportar soluciones innovadoras basadas en estándares de mercados abiertos; como el protocolo MQTT, etc. Así pues, desde este rol se busca garantizar principalmente la flexibilidad del sistema.

La necesidad de automatizar los procesos de control de calidad, empaquetado y paletizado surge en base a tres factores de riesgos; la salud laboral de los operarios, la eficiencia operativa y la trazabilidad digital.

Históricamente, las tareas de clasificación y descarte de piezas cerámicas rotas o con taras se apoyaban en la experiencia y visionado del operario; además de la capacidad física para transportar pesos durante una larga jornada laboral. Sin embargo, este enfoque presenta diversas limitaciones, como la fatiga visual o muscular de los trabajadores, riesgo de lesiones y variabilidad tanto en el rendimiento como en la eficacia del proceso. Es por ello por lo que la sustitución de estas tareas por unidades robóticas industriales permite eliminar el factor de riesgo humano, garantizar la repetibilidad de los procesos, la validación de trayectorias robóticas y optimización cinemática previo a realizar una inversión de capital.

El proceso de automatización e integración abarca desde el flujo completo de producción, manipulación robótica, supervisión del estado del producto cerámico y hasta finalmente, el registro de datos mediante una base de datos.

3. Propuesta de Automatización

3.1. Análisis de Alternativas

Para la optimización del final de la línea de la planta se llevó a cabo un estudio de viabilidad técnica y económica evaluando tres alternativas de automatización:

- Sistemas rígidos tradicionales: esta opción contemplaba el uso de brazos robóticos industriales clásicos en celdas completamente cerradas. Tienen la ventaja de su alta velocidad pero el problema principal es que por ley exigen un vallado físico de seguridad muy grande alrededor, lo que quita demasiado espacio en la planta de trabajo y aísla por completo el proceso.
- Automatización fija: la opción más barata para comenzar. Consistía en usar empujadores neumáticos y desviadores mecánicos para separar las piezas. El principal problema es que es una solución totalmente rígida. Ya que, si más adelante en el tiempo se quiere cambiar el tamaño de los azulejos o de las cajas, habría que parar la producción para poder recolocar los componentes mecánicos y recalibrar todo a mano.
- Robótica colaborativa: la opción elegida fue utilizar dos cobots de Universal Robots lo cual permite una fluida comunicación sin generar conflictos: un UR5e para coger y empaquetar los azulejos uno a uno, y un UR30 para mover las cajas pesadas al palé. Lo bueno de los cobots es que tienen sensores integrados que miden la fuerza y el esfuerzo; si chocan con un operario, se frenan en el acto. Esto permitiría eliminar las jaulas de seguridad ahorrando espacio. Además, como todo el control va por software en Python, cambiar de formato es tan fácil como modificar unas líneas de código.

3.2. Objetivos del Proyecto

Los objetivos del proyecto se han estudiado en base a las posibles formas de mejorar los aspectos: productividad y rendimiento, pero sin perder la calidad de clasificación y precisión del empaquetado del azulejo, ya que se trata de una automatización a nivel industrial, además de la flexibilidad y escalabilidad; y por último, la seguridad de la celda. Así pues, todos los objetivos son medibles, alcanzables y realistas mediante la implementación de esta propuesta.

En suma al tema de la definición de los KPIs asociados a cada criterio, a continuación se va a comentar cuales han sido los elegidos. Previamente, se va a definir el concepto de KPI para una mayor determinación de estos.

En primer lugar, el éxito de objetivos de productividad y rendimiento del proyecto, viene determinado por la capacidad del sistema automatizado para igualar o superar el ritmo de producción de la planta actual (trabajo manual) sin generar cuellos de botella. Para ello el parámetro a estudiar (KPI de productividad) ha sido el tiempo de ciclo. Se ha evaluado el tiempo total desde que un azulejo es detectado por la cámara Matrix Iris GTR hasta que la caja llena es posicionada por el UR30 en el palé de 800x800 mm.

Para obtener la diferencia entre los tiempos de ciclos del proceso de producción manual y la automatizada, se ha requerido del diseño de un diagrama de flujo del proceso manual y de la cronometración de la simulación del RoboDK.

En la fase de empaquetado de azulejos, el proceso manual requiere que el operario inspeccione visualmente cada pieza, decida su estado y la manipule en función de dicha decisión. Esta secuencia introduce tiempos muertos asociados a la toma de decisiones y a la variabilidad humana, además de duplicar esfuerzos en el caso de azulejos defectuosos o rotos, que deben ser retirados manualmente del flujo productivo. No obstante, el tiempo aproximado para la realización de esta tarea conlleva un total de 5 segundos. Eso implica que para el llenado de una caja de azulejos el tiempo oscila en torno a 50-60 segundos, puesto que la caja contiene en su interior un total de 10 azulejos.

En la fase de paletizado, el proceso manual presenta tiempos elevados asociados a la manipulación de cajas pesadas y a la espera de disponibilidad del palet. El operario debe comprobar de forma continua si la caja está completa y si el palet correspondiente dispone de espacio suficiente antes de proceder a su colocación. Estas comprobaciones se realizan de manera visual y requieren interrupciones constantes del flujo de trabajo. Es por ello por lo que el transporte de cada caja al palet supone alrededor de 10 segundos. Por lo tanto, el llenado de un palet que consta de 10 cajas, asciende a un proceso de duración de 324 segundos (5 minutos y 24 segundos).

No obstante, cabe recalcar que estos tiempos son aproximados, ya que en el proceso manual existe el factor humano. En otras palabras, las cifras obtenidas anteriormente son meramente orientativas, ya que muchas veces el tiempo podría ser mayor en base a la velocidad del operario -desde la zona de recogida de la caja hasta la dejada en el palet-. También, el factor humano incluye paradas técnicas dedicadas al descanso del operario o incluso momentos donde se requiere una parada del operario para recibir instrucciones técnicas por parte de otro operario. Todos estos aspectos comentados se engloban en el término definido por factor humano.

En definitiva, el tiempo de ciclo desde que llega un azulejo por la *Cinta A* hasta que se deposita la caja en el palet correspondiente asciende a 6 minutos y 24 segundos aproximadamente.

Después de analizar el proceso sin automatizar, se puede concluir que existen pérdidas de tiempo y un alto esfuerzo manual en las tareas de clasificación, empaquetado y paletizado. Por este motivo, se ha decidido automatizar el proceso con el objetivo de mejorar la eficiencia y reducir los tiempos de trabajo.

La automatización del proceso consigue recortar las pérdidas de tiempo y disminuir el esfuerzo manual de las tareas de clasificación, empaquetado y paletizado en gran medida. La primera parte del proceso, una vez automatizado, acarreará un tiempo total de 10 segundos. Por lo tanto, para el llenado completo de una caja de 10 azulejos acarreará un total de 100 segundos (1 minuto y 40 segundos). Mientras que el segundo proceso, implica alrededor de 7 segundos. Es decir, que en comparación con el proceso manual, éste tardará aproximadamente 2 minutos y 6 segundos para completar un palet compuesto por 18 cajas.

La puesta en marcha de la automatización implica que el tiempo de ciclo de 6 minutos y 24 segundos se reduzca casi a la mitad de tiempo, 3 minutos y 46 segundos. Siendo así una diferencia de tiempo de 2 minutos y 38 segundos entre ambos procesos.

DATOS DE PARTIDA				AHORRO AUTOMATIZACIÓN DEL PROCESO			
Jornada laboral			8 horas	Ahorro de tiempo total			0:02:38
Turnos			2 mañana/tarde	Tiempo total del proceso automatizado			0:03:46
Tiempo de ciclo PROCESO A			0:01:00	Ahorro nº de operarios			4
Tiempo de ciclo PROCESO B			0:05:24	Tiempo de ciclo automatizado PPROCESO A			0:01:40
Sueldo de un operario brutos anuales			25.000,00 €	Tiempo de ciclo automatizado PPROCESO B			0:02:06
Aumento de productividad			10%	Ahorro dinero operarios anuales			100.000,00 €
Tiempo de ciclo TOTAL			0:06:24				

No obstante, para el cálculo de los diferentes tiempos de ciclos del proceso automatizado hay que destacar dos objeciones al respecto; la primera es que con respecto al primer proceso no se ha tenido en cuenta el tiempo que tardan los azulejos en llegar a la zona de pick de la *Cinta A*, debido a que esta implementación también tiene constancia en el proceso manual, por lo tanto al tratarse de los mismo tiempos se ha obviado. La segunda objeción es en relación al segundo proceso. A la hora de contabilizar el tiempo de ciclo del segundo proceso no se ha añadido el tiempo transcurrido desde que la caja de azulejos sale del primer proceso hasta el final de la cinta mecánica que lo transporta. Esta fase alberga un total de 6/7 segundos en el proceso final. Como contrapartida, al igual que en el proceso manual no se ha añadido el tiempo extra a causa de los factores humanos, en este proceso tampoco se ha tenido en cuenta estos tiempos reducidos.

La segunda categoría a comentar es la calidad de clasificación y precisión de posicionado del azulejo. Se busca garantizar que la automatización no solo sea más rápida, sino también más exacta que la calidad humana. Es por ello que el parámetro a estudiar (KPI) ha sido la eficacia o también conocida como la tasa de error. Es decir, la lógica de control debe demostrar una tasa de éxito de al menos el 98% en la separación de piezas según su estado (correcto, defectuoso o roto), acorde a los estándares de la industria de la cerámica. El sistema integra una cámara Matrix Iris GTR diseñada para proporcionar la base tecnológica de clasificación, permitiendo en futuras actualizaciones la identificación precisa de defectos mediante análisis de imagen. Seguidamente, se ha analizado también el parámetro de la precisión, de manera que el sistema aproveche la repetibilidad de las unidades robóticas colaborativas, UR30 y UR5e, para asegurar que los azulejos se depositen sin colisiones laterales dentro de la caja. La automatización del proceso va a reducir la tasa de error gracias a la diferencia de capacidad de constancia y precisión entre un proceso automatizado y un

trabajador. Todos estos aspectos en conjunto permiten a la empresa reducir el porcentaje de errores y, por ende, no tener tantos desperdicios económicos.

Para el estudio de la flexibilidad y la escalabilidad, se ha analizado y mejorado el parámetro (KPI) del tiempo de reconfiguración. Este parámetro mide el tiempo necesario para adaptar la celda a un nuevo formato ya sea de azulejo o una variante en las distancias entre los objetos del espacio de trabajo. Así pues, en caso de ser necesaria una actualización, la organización de programas por carpetas, y la estructura robusta de la celda en frames, permite hacer las modificaciones necesarias sin tener que reescribir la lógica de todos los programas o reorganizar los objetos. Por ello, se considera un éxito que al modificar las dimensiones de la pieza el proceso se acopla automáticamente sin errores.

Y para finalizar, en cuanto a la categoría de seguridad del operario en el espacio de trabajo, se ha pretendido eliminar las exigencias físicas del entorno, así como la reducción de riesgos. Se busca validar que el hardware seleccionado pueda soportar el trabajo que previamente llevaban a cabo los operarios. Para el éxito de éste, se debe tener en cuenta dos parámetros; el parámetro de la gestión de carga de las unidades robóticas, más específicamente del UR30, puesto que es el cobot que se hace cargo del paletizado de las cajas de azulejos. Y el parámetro del alcance del robot en el espacio de trabajo. De este modo, se ha asegurado que no existan puntos con singularidades, es decir, puntos donde el robot se bloquee y no pueda llevar a cabo la tarea que está realizando y, además que alcance en este caso todas las esquinas del palé.

3.3. Otros Aspectos

La integración de la celda automatizada trasciende la mejora de los tiempos de ciclo, provocando una transformación integral en la operativa industrial. En primer lugar, se ha dotado a la celda de una infraestructura de visión artificial basada en la cámara Matrix Iris GTR, lo que proporciona la base tecnológica necesaria para garantizar la estandarización del control de calidad. Aunque el sistema de control actual se apoya en variables globales para asegurar la estabilidad del prototipo, esta implementación permite una evaluación mediante parámetros matemáticos constantes, eliminando la variabilidad intrínseca al factor humano -como la falta de concentración o la fatiga- y posicionando a la celda para una transición hacia la clasificación totalmente autónoma. A esto se suma la optimización del flujo de información mediante la arquitectura asíncrona basada en MQTT, que permite que los componentes de la celda operen de manera desacoplada. Esto significa que cualquier ajuste necesario en un cobot no bloquea el resto de la línea, maximizando la eficiencia global del sistema al evitar los paros en cadena que ocurrían en los procesos manuales antiguos.

En cuanto a los recursos humanos y materiales, la propuesta redefine el rol del operario, permitiendo su transición de una carga de trabajo físico repetitivo y potencialmente lesivo -manipulando cajas de hasta 20 kg- a funciones de supervisión técnica y mantenimiento. Esta evolución no solo dignifica el puesto de trabajo, sino que mejora la

retención de talento al alinear los perfiles con las demandas de la industria 4.0. Además, la eliminación de vallados físicos mediante el uso de cobots (UR5e y UR30) reduce los costes de infraestructura y permite una disposición más ágil del espacio en planta, facilitando un acceso rápido ante cualquier incidencia técnica.

Finalmente, la gestión y planificación del proyecto se ven beneficiadas por una mayor flexibilidad operativa. Al basar la configuración en estructuras de datos y *frames*, el cambio de formato de azulejo deja de ser un proceso mecánico complejo para convertirse en una actualización de software, permitiendo una capacidad de respuesta ante la demanda del mercado mucho más rápida. Esta agilidad se complementa con una mejora en la sostenibilidad, ya que los cobots ajustan su consumo eléctrico a la tarea real en cada instante, reduciendo la huella energética en comparación con la maquinaria tradicional de consumo constante, y garantizando una trazabilidad absoluta mediante la base de datos PostgreSQL, la cual asegura la integridad total del producto desde el origen.

3.4. Impactos adicionales

- **Ambientales:** Se logra una mayor sostenibilidad al reducir el desperdicio de producto cerámico, gracias a la delicadeza en la manipulación que ofrecen las garras de vacío frente al manejo humano.
- **Tecnológicos:** La implementación de estándares abiertos, como el protocolo MQTT, garantiza la escalabilidad del sistema y su futura integración con otros sistemas de gestión (ERP o MES) de la planta.
- **Logísticos:** La mejora en la uniformidad del apilado en los palets (800 x 800 mm) asegura que la carga sea compatible con los estándares de transporte europeos, optimizando el aprovechamiento del espacio en camiones.
- **Salariales:** La optimización de turnos y la reducción de costes de mano de obra directa genera un ahorro anual estimado de 100.000 €, lo que permite recuperar la inversión inicial en menos de dos años.

3.5. Límites y Restricciones del Proyecto

La implementación de la solución automatizada presenta una serie de condicionantes técnicos y operativos que han sido definidos para garantizar la viabilidad y seguridad del sistema en el entorno industrial:

- **Alcance físico y distribución espacial:** La configuración del layout está estrictamente condicionada por las dimensiones físicas de la nave y el alcance operativo de los brazos robóticos. Concretamente, el radio de alcance de 850 mm del UR5e y de 1300 mm del UR30 determina la disposición máxima permitida para las

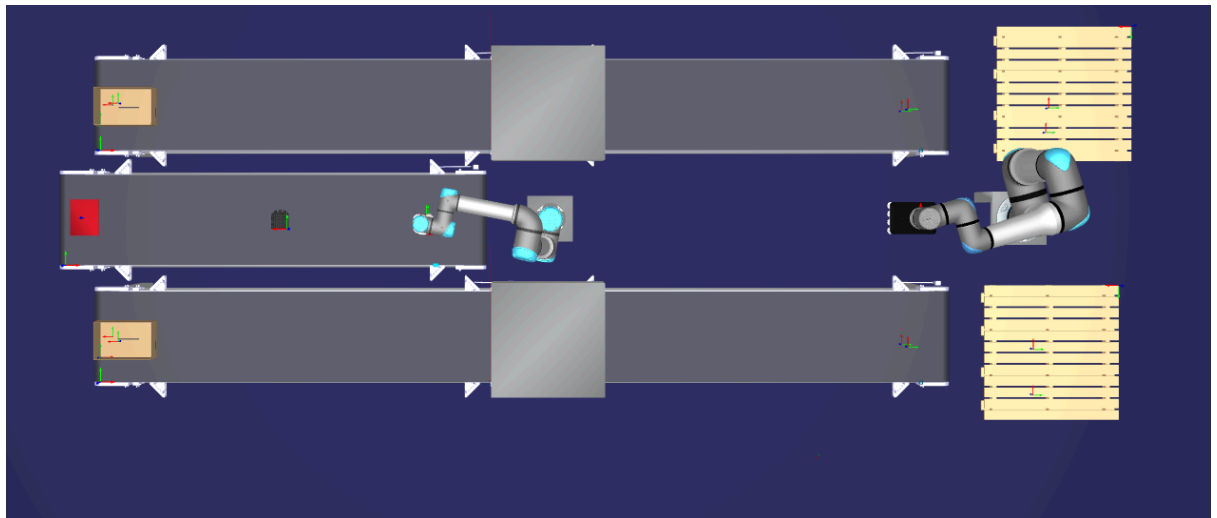
cintas transportadoras y las zonas de paletizado, limitando la flexibilidad en la reubicación de estos elementos dentro de la planta.

- **Capacidad de carga operativa:** El sistema de paletizado ha sido diseñado considerando una capacidad nominal de carga máxima de 30 kg, una especificación crítica establecida por el cobot UR30. En consecuencia, esta restricción impone un límite estricto al peso total de las cajas de azulejos, obligando a dimensionar el contenido y el empaquetado para no superar los umbrales de seguridad y repetibilidad del robot.
- **Flexibilidad ante cambios de formato:** Si bien el sistema destaca por su versatilidad gracias a la programación basada en software -lo que permite ajustar las trayectorias de forma ágil mediante código-, esta flexibilidad tiene un límite físico. Cualquier modificación sustancial en las dimensiones de los azulejos o del tipo de caja requeriría una reconfiguración mecánica manual de las cintas transportadoras, lo que implica una parada temporal del flujo de producción.
- **Dependencia del control por variables globales:** Debido a las particularidades del desarrollo del prototipo, se ha sustituido la implementación de un sistema de visión artificial autónomo por una gestión basada en variables globales. Esta restricción limita la capacidad del robot para identificar y clasificar defectos en las piezas de manera automática y en tiempo real, requiriendo que la distinción entre azulejos conformes y defectuosos sea gestionada externamente o mediante la intervención del operario, en lugar de ser un proceso totalmente autónomo integrado en la celda.

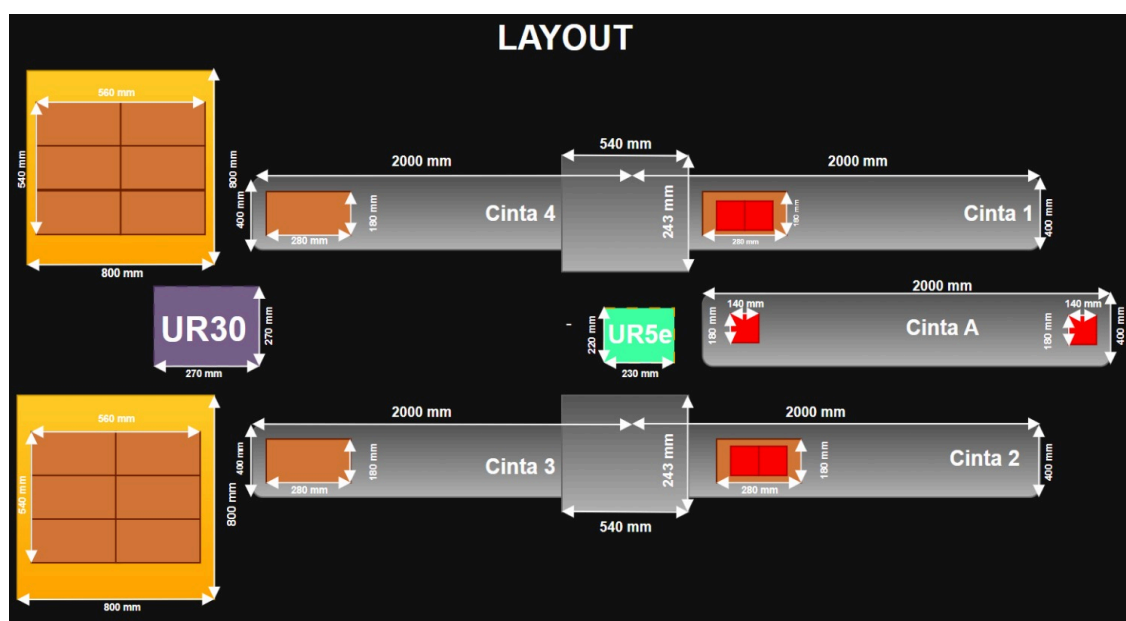
4. Diseño de la Solución

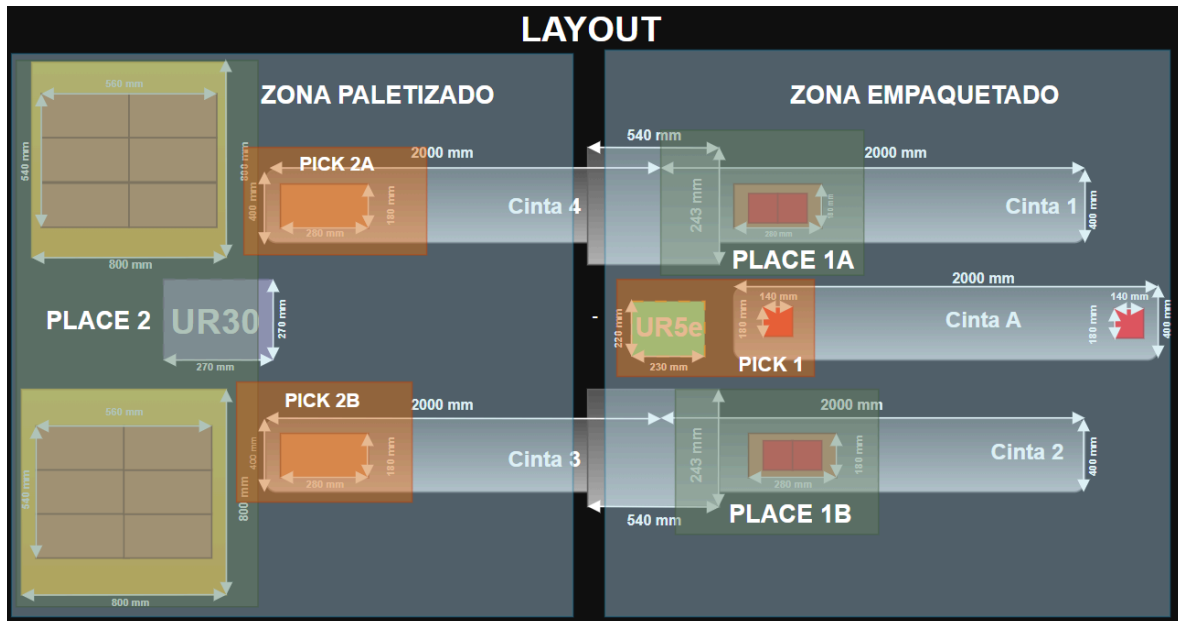
4.1. Elementos y Distribución del Espacio de Trabajo

La celda ha sido diseñada para garantizar un flujo de trabajo continuo, seguro y eficiente a la hora de transportar y empaquetar los azulejos de manera segura, garantizando así el buen estado de los mismos. Para ello, se han elegido cuidadosamente los elementos junto con sus características que conforman la celda. Para una justificación y descripción más detallada se ha optado por diseñar un layout que permita visualizar todos los elementos implementados de una manera más sencilla.

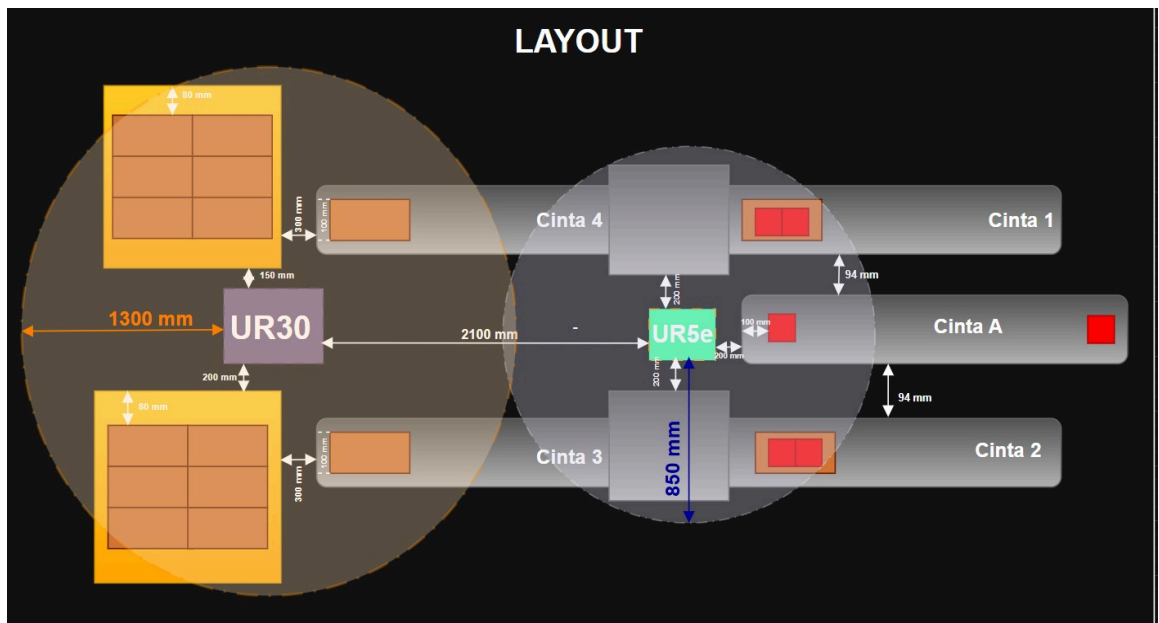


Esta sería la celda de trabajo desde una visión superior, pero como ya se ha mencionado anteriormente, la justificación de los elementos y de sus ubicaciones se basará principalmente sobre los layouts diseñados para una visión más sencilla y focalizada.





Estas imágenes representan las medidas de los elementos implementados, así como las medidas de los palés, la base de los cobots o la longitud de las cintas mecánicas y las dos zonas de trabajo, junto a sus respectivas zonas de pick & place de cada una de las fases del proyecto. No obstante, para una mayor especificación del proyecto se ha elaborado una segunda parte del layout que detalla más específicamente las distancias entre los objetos y los alcances de los cobots. Esto ayudará a una mejor comprensión de la explicación.



En primer lugar, cabe destacar que se ha decidido implementar robots colaborativos (cobots) como es el caso del UR30 y el UR5e en lugar de robots industriales convencionales porque al ser cobots, se elimina la necesidad de cerramientos físicos, también conocidos como jaulas de seguridad. Esto ayuda a reducir el presupuesto de la instalación y, a la vez permite a los operarios interactuar en el entorno como en el caso de tareas de supervisión de una forma segura y controlada. Además que este proyecto no requiere de robots que contengan una gran fuerza de carga porque los materiales a transportar o empaquetar son cargas menores, las cuales antes de la automatización la llevaban a cabo operarios. Asimismo, otro aspecto determinante para esta decisión ha sido la versatilidad de programación, puesto que a la hora de reprogramar trayectorias u otros cambios en la logística de la celda, es más fácil realizar el ajuste en robots colaborativos como éstos que en robots industriales.

Antes de continuar, se va a dividir el proyecto en 2 fases: la fase de empaquetado y control de calidad del azulejo, y la fase de la paletización de los azulejos.

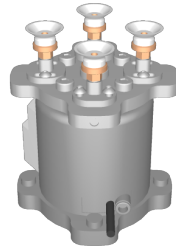
En cuanto a la primera fase, control de calidad y empaquetado de azulejos, se ha optado por implementar un robot colaborativo UR5e. Éste posee una carga máxima de 5kg, lo cual es ideal para el manejo de azulejos individuales de un tamaño 140 x 180 mm como es el caso del proyecto. En este caso, los azulejos de este tamaño pesan alrededor de 400-700 gramos, por lo que no supondrá ningún problema al UR5e a la hora de transportarlo de la cinta mecánica a la caja de cartón. Al trabajar el cobot dentro de su rango de esfuerzo óptimo, garantiza un buen cuidado en sus articulaciones, lo cual ayudará a una larga vida útil de éstas, además de permitir un trabajo mucho más eficiente.

Otro de los aspectos por los que se ha elegido este modelo de robot es su precisión. El UR5e tiene una repetibilidad de ± 0.03 mm lo cual garantiza la proximidad entre las posiciones alcanzadas por el efector final después de aplicar el mismo movimiento un número determinado de veces desde la misma dirección y con las mismas condiciones de carga y velocidad. En otras palabras, esta cifra asegura una consistencia mecánica excepcional en procesos repetitivos como este. Esto significa que el UR5e es capaz de moderar con un porcentaje de error ínfimo la variabilidad posicional, garantizando de esta manera que el margen de error en tareas críticas, como es el empaquetado de piezas tan frágiles como los azulejos, sea el mínimo posible.

En cuanto a su ubicación, se ha colocado a 200 mm de las cintas mecánicas que tiene alrededor para que no tenga problemas a la hora de coger y depositar el azulejo. El UR5e cuenta con un alcance de 850 mm, lo cual simplificará en gran medida la complejidad de empaquetar el azulejo, ya que permite cubrir todo el área de trabajo necesaria sin necesidad de alcanzar su máxima posición. Se ha ubicado a una distancia tan corta de las cintas mecánicas en gran parte por uno de los aspectos anteriormente comentados; al situar la base del UR5e a una distancia corta, el brazo puede trabajar en su zona de confort sin necesidad de que éste se fuerce para intentar alcanzar posiciones que se encuentran en su rango límite. De esta manera, se consiguen movimientos más fluidos y reduce el desgaste en las articulaciones del mismo.

Por último, uno de los aspectos más importantes que ha ayudado a la decisión de por qué implementar este cobot en el proyecto han sido sus grados de libertad. A primera vista, este proceso parece necesitar solamente 4 grados de libertad, sin embargo esto limitaría a movimientos de los ejes X, Y, Z y una rotación vertical. En cambio, se ha optado por la implementación de un cobot que tenga 6 ejes porque a menudo los azulejos que vienen por la *Cinta A* vienen con orientaciones diferentes que obligan al robot a ejecutar una rotación de muñeca. También, en el proceso de encajar los azulejos dentro de la caja hay veces que se requiere de ligeras inclinaciones para entrar correctamente y evitar colisiones con la caja que podrían ocasionar desperfectos en el producto. De igual manera, al disponer de más articulaciones, el UR5e posee una mayor redundancia cinemática que supone una ventaja en el proyecto. Esto permite poder articular de muchas formas diferentes sus articulaciones para esquivar elementos que conforman la celda y evitar colisiones y posibles problemas en el proceso.

A continuación, para el agarre de los azulejos se ha decidido utilizar la garra SMC ZXP7A01-ZP20U-X1 Vacuum Gripper.



Esta garra está formada por 4 ventosas situadas a los laterales. Esta geometría permite agarrar los azulejos con una mayor firmeza.

No obstante, si por la *Cinta A* viene un azulejo roto, la cámara Matrox Iris GTR será la encargada de detectarlo. En teoría esta cámara sirve para diferenciar el estado del azulejo y poder distinguir entre los azulejos en buen estado y los que sirven para una venta más económica. Pero debido a las circunstancias del proyecto, no ha podido ser posible la programación e implementación de este elemento, por lo que se ha decidido diferenciar los azulejos según una variable global en el programa. Es por ello mismo por lo que la cámara no aparece en el layout, debido a su presencia decorativa nada más.

Una vez la cámara lo ha detectado, ésta enviará esa información al robot de manera que éste no llevará a cabo el pick, sino que el azulejo seguirá hasta el final de la cinta mecánica hasta caer en la papelera. La papelera no ha sido incluida en el layout, pero ésta se encuentra entre la base del robot UR5e y la cinta mecánica.

Por otra parte, esta garra es ideal para el proceso por la misma razón que el robot, porque tiene una carga útil de hasta de 7 kg lo cual garantiza aún más la sujeción del azulejo de la manera más firme posible para garantizar su máxima calidad. También, se ha elegido una garra que cuenta con ventosas y no otra garra convencional porque es la herramienta que

mejor se adapta al producto debido a su superficie, ya que permite el agarre por la cara superior sin dañar los bordes y además, ese agarre no interfiere a la hora de dejar el azulejo en la caja. En cuanto a la ubicación de la garra, ésta se encuentra al extremo del brazo UR5e.

A continuación, una vez comentado el brazo robótico y la herramienta con la que se va a acoplar al producto, es importante mencionar las dimensiones de los azulejos y el tamaño elegido de las cajas con el fin de la mayor eficiencia de espacio posible.

Para ello, previamente se debe aclarar los motivos por los que se ha decidido implementar 5 cintas mecánicas de la misma longitud en el proyecto. Cada cinta mecánica consta de una longitud de 2000 mm y una anchura de 400 mm, esto responde a la necesidad de optimizar el flujo de los azulejos y garantizar la mayor estabilidad del proceso de empaquetado. Una longitud de 2 metros permite acumular una recta fila tanto de azulejos como de cajas vacías esperando a llenarse y de cajas llenas esperando a ser paletizadas. De igual modo, cabe destacar que al final de cada cinta mecánica se encuentra el sensor SICK WL4S Laser Sensor que controla la presencia o la no presencia de piezas.

Este sensor a diferencia del resto de sensores infrarrojos utiliza un láser rojo altamente enfocado. Esto supone una ventaja porque genera un spot de luz muy pequeño y muy nítido, lo que permite una detección de bordes extremadamente precisa.

Luego, vale la pena señalar que los azulejos son un producto que suele tener un acabado brillante o incluso esmaltado -dependiendo del tipo- que pueden llegar a causar reflejos falsos en sensores ópticos. El SICK WL4S está diseñado precisamente para ignorar aquellos reflejos y detectar con la misma fiabilidad tanto el brillo del azulejo, como la superficie de las cajas de cartón.

Cada vez que el sensor WL4S detecta un elemento, es decir, que de alguna manera se corta el haz láser entre el sensor y el reflector, se trate ya sea de un azulejo o una caja, éste envía una señal de entrada (DI) para que la cinta mecánica se pare. Para reactivar la continuación del proceso, el sensor WL4S debe dejar de detectar el elemento, es decir, la cinta mecánica se queda parada hasta que el elemento no haya sido retirado. Una vez retirado el azulejo o la caja se repite el proceso de manera repetitiva.

Por otra parte, volviendo a las cintas mecánicas, la anchura de éstas dota de estabilidad a las cajas y a los azulejos, puesto que consta de espacio de sobra y de velocidades suaves, lo cual ayuda a que los elementos se estabilicen mecánicamente.

En cuanto a la ubicación de éstas se puede observar que la *Cinta A* se sitúa enfrente del UR5e. Esto es debido a que esa disposición facilita en gran medida que el robot pueda agarrar de la manera más cómoda y segura los azulejos, para luego depositarlos en las cajas que se encuentran en las cintas mecánicas a los lados (*Cintas 1 y 2*). Exactamente colocadas a 94 mm de la *Cinta A*, esta proximidad permite que el robot trabaje en su rango cómodo sin necesidad de forzarse. Luego, las cintas mecánicas transportan las cajas ya llenas a la siguiente fase, la fase de paletizado. Una vez llegan al final de las *Cintas 3 y 4* los sensores WL4S detienen la cinta hasta que no se retire y se paletice la caja. En este caso el robot UR30

no se sitúa de igual manera que el robot UR5e, ya que en este caso los elementos a paletizar no vienen de la misma cinta como ocurría en la *Cinta A*. Sino que vienen de cintas diferentes debido a la distinción de calidad entre los productos, es por ello que el robot se situará en medio de las cintas mecánicas, a una distancia aproximadamente de 950 mm. De manera que su amplio rango le permitirá trabajar tanto en la parte superior (*Cinta 4*), como en la parte inferior (*Cinta 3*).

Una vez se ha comentado la disposición de las cintas mecánicas, a continuación se va a abordar el tamaño de los azulejos y de las cajas.

Así pues, en cuanto a los azulejos, éstos cuentan con un tamaño de 180 x 140 mm. Se ha elegido este tamaño de azulejo porque se trata de un tamaño estándar en la industria de los azulejos. Este tamaño a diferencia del resto es de los más solicitados por los clientes, por lo que conlleva una mayor producción y a la vez tiene como consecuencia una mayor necesidad de automatización del proceso de empaquetado y paletizado para poder abarcar la demanda del consumidor. Por otra parte, el tamaño es ideal para ubicar el azulejo en medio de la cinta mecánica, lo cual implica un menor riesgo de que el azulejo sufra algún daño en su revestimiento.

De esa manera, es por ello por lo que el tamaño de la caja dependía directamente de la proporción del azulejo. Una vez se ha determinado el tamaño del azulejo, hay que determinar el tamaño de la caja donde se va a empaquetar. En este caso, se ha decidido que las cajas van a ser de un tamaño de 180 mm de ancho, 280 mm de largo y 120 mm de alto. De esta forma, permite apilar 2 columnas de azulejos dejando un espacio justo por el margen de error tanto del robot como del tamaño del azulejo, y un total de 5 filas, es decir, en total cada caja contendrá un total de 10 azulejos. Con respecto a esa distribución, cabe recalcar que es la distribución que suele ser implementada de manera más habitual en la industria del azulejo.

En cuanto a la ubicación de las cajas en la celda, en la primera fase las cajas se encuentran vacías y se pararán al final de la *Cinta 1 y 2* esperando que el robot UR5e termine de llenarlas. Al igual que en el caso de los azulejos, la cinta se detendrá cuando la caja se sitúe a 100 mm del final de la cinta. Una vez que las cajas están llenas, éstas avanzarán y pasarán a las siguientes cintas, dependiendo del tipo de azulejo que transporten. Es en este momento cuando se trata de la fase 2 del proyecto. En esta fase el funcionamiento para las cajas es el mismo, avanzan hasta llegar al final de la cinta donde el sensor WL4S detectará que hay objeto y detendrá la cinta hasta que la caja no haya sido retirada y colocada en el palé adecuado. En ambas fases, la ubicación de la caja es en la zona central de la cinta mecánica por la misma razón que los azulejos. De esa manera, obtienes una mayor estabilidad y evitas riesgos de que la caja pueda caerse y, por ende, dañar el contenido que contiene dentro.

Para continuar con el orden de la fase 2, cabe explicar a continuación qué tamaños de palets se han utilizado y, posteriormente se explicará los motivos por los que se ha implementado el robot UR30 y su herramienta.

Para el transporte de los azulejos se han implementado dos palets; uno para el paletizado de los azulejos en buen estado y el otro palé para aquellos azulejos que contienen

algún defecto en su diseño. Ambos palets son de tamaños iguales, aproximadamente 800 mm x 800 mm. Este tamaño permite a la empresa paletizar un total de 6 cajas (apilables a 3 alturas) de manera totalmente segura, ya que el tamaño del palé permite dejar una separación de 80 mm de los bordes. Así pues, las cajas que contienen los azulejos en su interior se situarán en la zona central de éste, disminuyendo así el riesgo de que si hay alguna colisión en el camino, el contenido tenga la mayor probabilidad de llegar íntegro al consumidor. Un centro de masas agrupado como se trata de las cajas mejora la integridad de la estructura.

En adición a esto, se ha implementado un palé de tamaño 800 x 800 mm porque se trata de unos de los estándares Europeos, este tipo de palé se conoce técnicamente como un palé cuadrado de base tipo “media paleta”, pero albergando una mayor profundidad. Al tener este tamaño permite que sea perfectamente compatible con el resto de camiones de Europa, los cuales tienen un ancho útil alrededor de 2400 mm.

En cuanto a su ubicación, éstos se sitúan entre 150 y 200 mm del robot UR30. En este caso sí que la ubicación dificulta en los casos extremos como las cajas que se encuentran en la parte superior del palé de la *Cinta 4* y el caso de las cajas que se encuentra en la parte inferior del palé de la *Cinta 3* del paletizado de las cajas. Sin embargo, aunque en este caso el robot sí que va a tener que llegar a localizaciones límite, es completamente necesario porque de esta manera el robot puede coger las cajas de las cintas y depositarlas en el palé, salvo el caso de 2 cajas puntuales las cuales sí podrá depositar pero con más dificultad que el resto. De cualquier otra manera, si se optara por otra ubicación el robot no podría alcanzar ciertos puntos del palé y se trataría de una situación peor a la actual.

Y por último, para concluir con la fase 2 y, por ende, con el proceso, se va a abordar a continuación los motivos por los que se ha decidido implementar el brazo robótico UR30, junto a su herramienta OnRobot VGP20 Vacuum Gripper.

En relación al cobot UR30, dicha elección para la fase final del proyecto no ha sido casual, sino que la implementación de éste responde directamente a la necesidad técnica de alcance y seguridad sin tener que implementar un robot industrial con todo lo que eso conlleva. En adición con esto, la razón principal de su elección principalmente ha sido su capacidad de carga de 30 kg (incluso con la capacidad de extenderse a 35 kg bajo ciertas condiciones).

Esta razón es sumamente importante porque una caja llena de azulejos puede llegar a ser considerablemente pesada, incluso llegando a alcanzar los 20 kg en algunos casos en la industria. De igual importancia, el paletizado es una tarea que exige cubrir áreas grandes tanto verticalmente como horizontalmente, por lo que esto en parte obliga al proyecto a implementar un robot que sea capaz de cargar con cajas pesadas y tener un gran alcance. En el caso del proyecto, el UR30 posee un alcance total de 1300 mm, lo cual permite cubrir el área de paletizado en la zona tanto de las cintas como en los palés. Sin embargo, si se hubiera tomado la decisión de implementar un robot con menor alcance en parte obligaría a colocar el palé excesivamente cerca de la base del robot y surgirían problemas de “zonas muertas” donde el robot no podría trabajar debido a la excesiva proximidad. Además, del alto riesgo de

colisiones a la hora de trabajar. Por consiguiente, el radio de 1300 mm le permite alcanzar ambas líneas de entrada (*Cintas 3 y 4*) y los puntos de dejada del palé sin necesidad de implementar una segunda unidad robótica, lo que reduce drásticamente el coste del proyecto y su complejidad.

En el caso de la precisión de la unidad robótica UR30 se observa que tiene una repetibilidad de ± 0.1 mm lo que garantiza que las cajas puedan quedarse de manera totalmente alineada con el palé. Esto es muy importante, ya que en el caso de la situación de que haya una columna mal alineada de azulejos podría provocar un colapso y, por ende, una pérdida económica de gran valor para la empresa que implemente el proyecto, además de la seguridad de los operarios.

Por lo que respecta a los grados de libertad, el UR30 cuenta con 6 grados de libertad. Al tener 6 ejes, el robot tiene múltiples configuraciones para llegar al mismo punto, lo cuál es sumamente determinante sobre todo para las 2 cajas situadas a los extremos donde el robot llegará haciendo un esfuerzo sobre sus articulaciones, puesto que se sitúan al borde del rango de su alcance. De la misma manera, esta característica es vital también para evitar colisiones con obstáculos como las propias cintas mecánicas o con las cajas que ya han sido apiladas anteriormente en el palé. En adición a esto, el robot a la hora de depositar las cajas con azulejos en las posiciones extremas necesita estirar el brazo mientras que a su vez mantiene la herramienta en horizontal. Este equilibrio simplemente puede ser garantizado por un robot de 6 ejes el cual pueda cumplir con el seguimiento de las múltiples trayectorias.

Por lo que respecta a su ubicación en la celda esta distribución se ha implementado con el fin de ahorrar en costes principalmente.

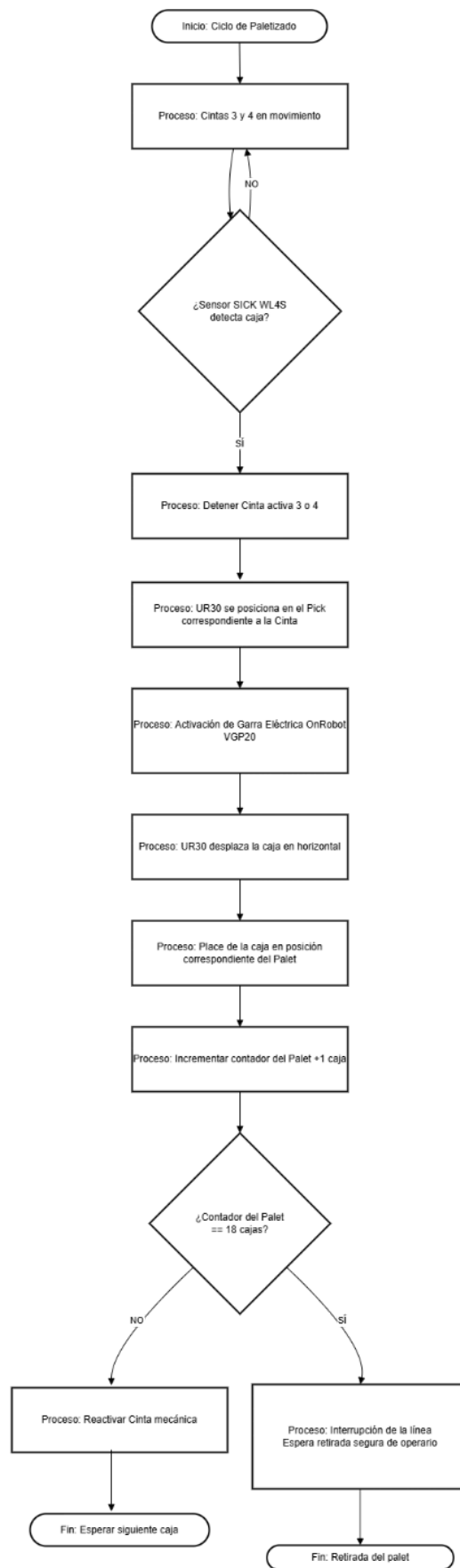
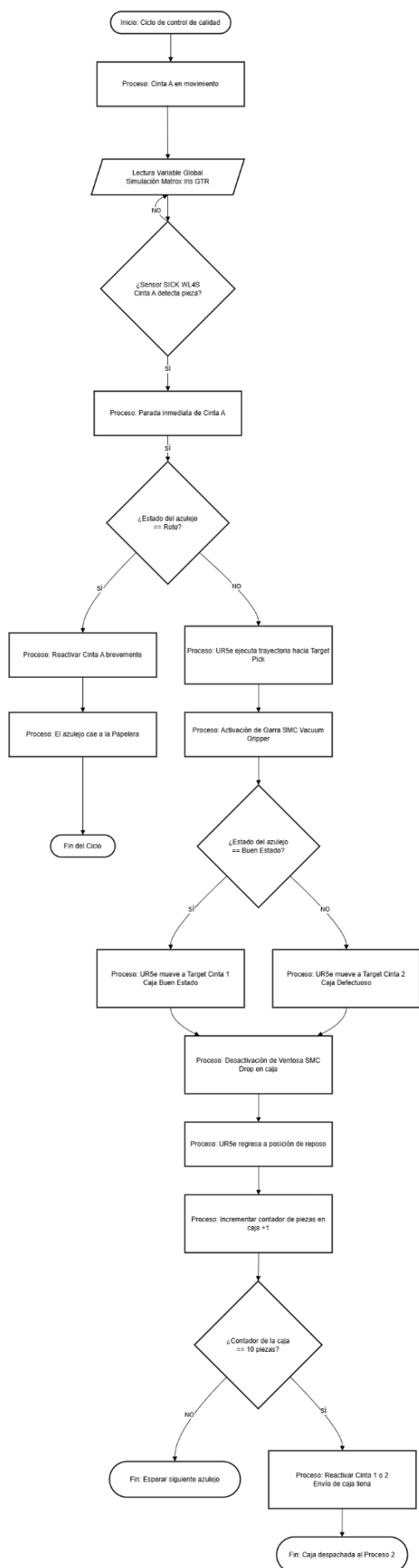
Finalmente, al extremo de la unidad robótica UR30 se sitúa la herramienta OnRobot VGP20 Vacuum Gripper.



La elección de la OnRobot VGP20 Vacuum Gripper se trata de una garra de vacío eléctrica que consta con una capacidad de carga hasta de 20 kg. Esto mismo permite manipular cajas de azulejos pesados manteniendo un margen de seguridad de 10 kg para el propio peso de la garra y los esfuerzos que realiza el robot a través de los movimientos acelerados. Se ha optado por una garra de vacío eléctrica porque a diferencia de las ventosas tradicionales, la VGP20 es 100% eléctrica, lo que conlleva de cierta manera un ahorro en costes, pero, sobre todo, evita a la hora de la instalación problemas de gestión de cables que podrían interferir en los 6 grados de libertad de la unidad robótica.

En relación con la herramienta, ésta contiene una peculiaridad y es que permite controlar de forma independiente sus canales de succión, lo cual es vital en este proyecto, puesto que se pueden activar solamente las ventosas de los extremos para asegurar que la caja no sufra ningún balanceo por los movimientos del robot. En relación a la superficie de la caja, se trata de una superficie porosa y a veces, irregular aunque no suele ser lo habitual, esta herramienta es capaz de ajustar su flujo de aire lo que garantiza un agarre firme incluso tratándose de superficies como estas. Por último, la estructura de la VGP20 está diseñada para entornos industriales robustos donde la presencia del polvo es algo habitual. Al ser compacta ayuda a que el UR30 pueda alcanzar sus puntos máximos de alcance siendo un mayor apoyo en la tarea del place en el palé.

Una vez definida la distribución espacial de los elementos que conforman la celda de trabajo, es fundamental para un mayor entendimiento del proyecto la definición de un plano lógico que implemente la sincronización de movimientos, el tratamiento de las señales de los sensores y la toma de decisiones algorítmicas.



4.2. Estrategias y Algoritmos de Programación Avanzada

En el diseño del software para la automatización de la celda, especialmente en el firmware del sistema embebido (ESP32-S3), se ha priorizado la fiabilidad, la escalabilidad y el rendimiento en tiempo real. Para garantizar que el control de la maquinaria sea robusto frente a las peculiaridades de las redes industriales, se han evaluado e implementado estrategias de programación avanzada y estructuras de datos eficientes.

4.2.1. Algoritmo de Hashing y Búfer Circular

En entornos industriales IoT basados en protocolos de publicación/suscripción como MQTT, es habitual enfrentarse a problemas de inestabilidad de red, retransmisiones del broker o rebotes de hardware que pueden provocar la llegada duplicada de un mismo comando en una ventana corta de tiempo. A modo de solución, se ha diseñado un mecanismo de deduplicación basado en una función Hash no criptográfica que procesa la concatenación del topic y el payload entrante, almacenando el resultado en un búfer circular (estructura FIFO de tamaño estático).

El procesamiento de un mensaje duplicado en esta celda podría resultar crítico, provocando un doble conteo de azulejos en la base de datos o activando falsos movimientos en las unidades robóticas. Al utilizar un algoritmo Hash, se convierten cadenas de texto de longitud variable en identificadores numéricos únicos, lo que permite realizar comprobaciones de forma extremadamente rápida. La elección de un búfer circular asegura que la complejidad espacial se mantenga estrictamente en $O(N)$ (siendo N el tamaño del búfer), evitando la fragmentación de memoria (Heap) en un microcontrolador con recursos limitados como la ESP32-S3 y garantizando un coste computacional constante y predecible.

4.2.2. Estructuras de Datos Compuestas

En cuanto a la gestión de las señales físicas (botones de spawn y vaciado de palets), se ha diseñado una arquitectura basada en matrices de estructuras de datos (Structs). Esta aproximación agrupa bajo una misma entidad lógica todos los atributos de un periférico: su pin físico, su estado de rebote temporal, y su metadato de red (el topic y payload MQTT asociado a dicha acción).

Evita la redundancia de código y el uso de múltiples variables globales desconectadas entre sí. Desde el punto de vista del diseño de software, esta estructura otorga una escalabilidad inmediata al sistema. Si en el futuro la celda requiere integrar nuevos sensores o botones para diferentes tipos de azulejos, el esfuerzo de programación es mínimo: basta con instanciar un nuevo elemento en la estructura de datos sin necesidad de alterar la lógica de control principal (bucle iterativo).

4.3. Diseño de la Base de Datos

En lo que respecta a los datos, con el fin de controlar la información generada en la celda automatizada, se ha optado por el diseño de una base de datos relacional (SQL), específicamente implementada bajo el motor de PostgreSQL. La elección de un modelo estructurado responde a la naturaleza estrictamente vinculante de los datos industriales generados en este proyecto. En un entorno de manufactura, es imprescindible garantizar la integridad referencial (por ejemplo, asegurar que un azulejo empaquetado pertenezca a un lote válido y que dicho lote esté asociado a un pedido real), por lo que las propiedades de consistencia, aislamiento y durabilidad de las bases de datos SQL resultan idóneas para el control de inventario.

El modelo conceptual se ha diseñado con el objetivo de proporcionar una trazabilidad absoluta, abarcando desde el origen de los materiales hasta el producto final. Para lograrlo, la arquitectura de datos se articula en torno a las siguientes entidades clave:

- **Trazabilidad del Producto (Azulejo y Cajas):** La simulación rastrea cada unidad de producto. La entidad *Azulejo* registra el número de serie y el estado de calidad asignado por el control automatizado (bueno, defectuoso o roto). A su vez, estos elementos se asocian a una *Caja_llena* (identificada por lotes y categorías de calidad), permitiendo conocer exactamente qué piezas contiene cada paquete paletizado.
- **Gestión Comercial (Clientes y Pedidos):** Para vincular la producción con la salida comercial, se han modelado entidades que asocian los lotes de cajas producidas con los pedidos realizados por los clientes, garantizando que el empaquetado responda a la demanda registrada.
- **Cadena de Suministro (Proveedores y Compras):** Se ha incluido el control sobre la entrada de material logístico, como las cajas vacías (*Caja_Vacia*), asociando su adquisición a proveedores específicos para controlar los costes de producción.

Desde la perspectiva de la Industria 4.0, este diseño convierte las acciones físicas de la planta en información estratégica. Cada vez que el sistema embebido (ESP32-S3) clasifica una pieza y la unidad robótica (RoboDK) ejecuta los movimientos de empaquetado y paletizado, la aplicación central gestiona el registro inmediato de estos eventos en la base de datos. Esta persistencia en tiempo real es lo que permite alimentar los cuadros de mando y calcular los KPIs críticos del proyecto.

4.4. Arquitectura y Diseño de una solución de Integración

Para lograr la sincronización en tiempo real entre el hardware físico, la lógica de control y el registro de datos, se ha diseñado una arquitectura de integración basada en el protocolo MQTT (Message Queuing Telemetry Transport). La elección de este estándar de

mensajería bajo el patrón de publicación/suscripción responde a su ligereza, baja latencia y alta fiabilidad, características fundamentales para las comunicaciones en entornos industriales.

El núcleo de la comunicación recae sobre un broker MQTT externo (broker.emqx.io operando por el puerto 1883), el cual actúa como intermediario central. La arquitectura establece un flujo de datos bidireccional, asíncrono y desacoplado.

En primer lugar, un flujo ascendente, es decir, el microcontrolador ESP32-S3 actúa como cliente publicador. Al detectar la interacción física con la botonera, el sistema embebido publica mensajes con payloads específicos en los canales designados. Una aplicación central, desarrollada en Python, está suscrita a estos mensajes; al recibirlos, registra el evento correspondiente en la base de datos PostgreSQL (asegurando la persistencia) y gestionando los movimientos mecánicos correspondientes.

En segundo lugar, un flujo descendente. En este caso, la aplicación central monitoriza el progreso del empaquetado y el nivel de llenado de los palets virtuales. Cuando un palet alcanza su capacidad máxima o el sistema altera su estado de funcionamiento, la aplicación publica mensajes de estado. El ESP32-S3, suscrito a dichos canales, interpreta el payload y actúa sobre las salidas físicas (encendido/apagado de LEDs de advertencia), cerrando así el ciclo de retroalimentación hacia el operario.

De este modo, para estructurar el tráfico de mensajes de forma escalable y evitar colisiones, se ha diseñado una jerarquía de topics bajo el espacio de nombres principal “*sim/working/*”. Los documentos de intercambio se han simplificado al máximo, utilizando caracteres numéricos o cadenas cortas (“on”, “off”) para minimizar el consumo de ancho de banda.

La taxonomía definida es la siguiente:

- **Eventos de Control de Calidad (Publicados por ESP32, Suscritos por RoboDK):**
 - *sim/working/button/spawn*: Notifica la entrada de un nuevo azulejo a la cinta. Intercambia los payloads “1” (Buen estado), “2” (Malo) o “3” (Defectuoso).
- **Eventos de Logística (Publicados por ESP32, Suscritos por RoboDK):**
 - *sim/working/button/empty*: Notifica la retirada y reposición manual de un palet lleno. Intercambia los payloads “1” (Palet 1 vaciado) o “2” (Palet 2 vaciado).
- **Señales de Estado Operativo (Publicados por RoboDK, Suscritos por ESP32):**
 - *sim/working*: Recibe comandos (“on” / “off”) para controlar el LED principal de funcionamiento de la planta.
 - *sim/working/palet1* y *sim/working/palet2*: Reciben comandos (“on” / “off”) para activar los indicadores led que alertan al operario de que las zonas de carga han alcanzado su límite de cajas.

Esta solución de integración indirecta a través de colas de mensajería garantiza que el microcontrolador, la base de datos relacional y el entorno de simulación operen de forma totalmente desacoplada, permitiendo futuras ampliaciones (como añadir nuevos sensores u otros clientes analíticos) sin necesidad de reescribir la lógica de red existente.

5. Desarrollo e Implementación

5.1. Gestión del Proyecto

5.1.1 Plan detallado e implementación del proyecto de asignatura

El proyecto de automatización de control de calidad, empaquetado y paletizado de azulejos se ha desarrollado bajo un plan de desarrollo estructurado en diferentes fases: análisis y definición, equipamiento y layout, programación, incorporación del microcontrolador ESP32-S3, documentación, preparación del proyecto y por último, la fase de montaje y exposición del proyecto.

Previo a desarrollar más exhaustivamente las diferentes fases del proyecto es importante explicar las diferentes funcionalidades que se han aplicado para garantizar una mayor organización en Trello.

En cada fase del proyecto se han añadido múltiples tarjetas que representan las diferentes tareas a realizar. No obstante, en todas las fases existe una tarjeta uniformemente verde denominada “*Tareas Completadas*”. Esta tarjeta cumple con la finalidad de ser el encabezado de todas aquellas tareas que se han marcado como completadas. En otras palabras, cada tarjeta que representa una tarea se ha marcado como completada, gracias a la programación interna que se ha implementado en a través de la opción que ofrece Trello, se sitúa de manera automática en la parte inferior de la tarjeta *Tareas Completadas*. Sin embargo, si una tarea que ya había sido marcada como completada se desmarca, del mismo modo, se sitúa en la parte superior de la tarjeta *Tareas Completadas* mostrando que esa tarea no se encuentra finalizada. Seguidamente, se ha añadido como complemento a la programación un elemento visual en la cabecera de cada tarjeta. En base al estado de la tarea, su encabezado obtendrá un color u otro; verde para las tareas completadas que a su vez se sitúan a continuación de la tarjeta de *Tareas Completadas*, roja si la tarea no ha sido iniciada y, como última opción, naranja para aquellas tareas que en su efecto práctico han sido marcadas en un primer momento como tarea finalizada pero posteriormente ha sido desmarcada, lo cual indica que esa tarea se encuentra en proceso.

En relación a las fases del proyecto, la primera fase que se ha desarrollado ha sido la fase que hace referencia al análisis y definición del mismo. Dentro de esta fase se ha buscado analizar las diferentes opciones de procesos a automatizar y con ello, su elección y definición. Previa a la definición del proyecto a automatizar, se ha realizado un estudio de la viabilidad del proyecto, de esta forma se obtienen datos que verifican los resultados esperados y el margen de tiempo que va a necesitar la empresa que aplique la automatización para recuperar

el desembolso de capital inicial. En el caso de que el análisis realizado indique que el proyecto es viable y la empresa recuperará su inversión en un plazo menor a dos años, se procederá a la planificación de éste.



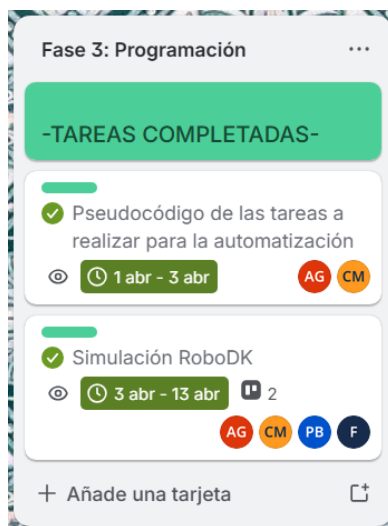
Al tratarse de la primera fase del proyecto, las fechas impuestas corresponden al inicio de los plazos asignados para el desarrollo de la automatización. De igual manera, al tratarse de la definición del proyecto se ha requerido de todos los integrantes de la consultora externa, ya que cada integrante está especializado en un ámbito diferente y puede aportar ideas y/o soluciones diferentes para el desarrollo del proyecto.

Después de la definición del proceso se desarrolla la segunda fase: equipamiento y layout. Para llevar a cabo el proyecto, el siguiente paso a realizar es la elección de todos los componentes y sus respectivas funcionalidades que se van a implementar en la planta de trabajo. Asimismo, después de la elección de los elementos que conforman la celda se procederá a la distribución espacial de éstos teniendo en cuenta los diferentes alcances de las unidades robóticas y su interacción con el resto de elementos. Una vez se ha definido la distribución espacial definitiva de los elementos, se prepara el entorno a utilizar.



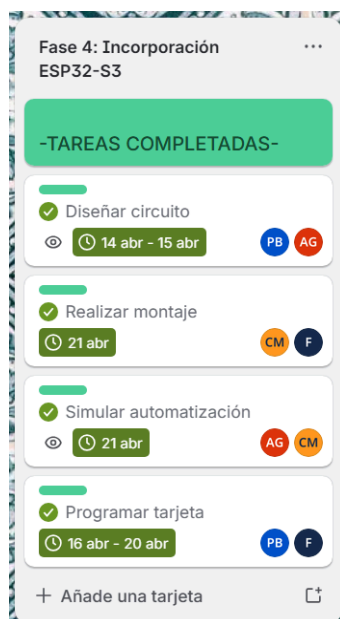
Conforme a las fechas establecidas, al tratarse de la segunda fase del proyecto las fechas son contiguas a las definidas en la fase uno. Y como en el caso anterior, la responsabilidad de las tareas recae en todos los integrantes, puesto que se tratan de conceptos generales y básicos para el desarrollo del resto de aspectos del proyecto.

La siguiente fase es la fase focalizada en la programación principalmente de la simulación de la celda de trabajo en RoboDK. Previo a la simulación se realizó un pseudocódigo como boceto para el desarrollo del código final.



Cabe destacar que la simulación de RoboDK se ha distribuido uniformemente entre todos los integrantes de la consultora externa, ya que la simulación abarca muchos ámbitos y consigo, dificultades. Por lo que en el caso de haber definido menos responsables en la tarea habría generado una excesiva carga de trabajo. No obstante, aunque la licencia solo se la atribuyó a un integrante, el resto ha hecho uso de la prueba gratuita.

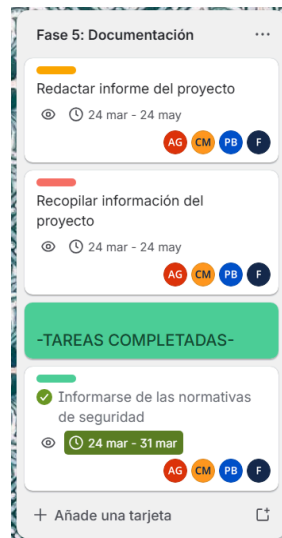
Seguidamente de la fase de programación se encuentra la cuarta fase, cuya finalidad es la incorporación del sistema embebido ESP32-S3. Principalmente la ESP32-S3 va a sustituir la función de la cámara de visión artificial que se encarga de la clasificación de los azulejos en base a su estado. Para ello, se ha requerido del diseño del circuito, ya que se ha implementado un sistema de botones que representan los diferentes estados -bueno, roto o defectuosos-, junto a los botones de retiradas de los palets llenos. Una vez se ha diseñado el circuito, se procede a realizar su correspondiente montaje y simulación para verificar su funcionamiento. Para el funcionamiento de la integración se asume que previamente se debe programar la tarjeta.



Con respecto a la distribución de las tareas, al tratarse de una fase compleja se ha optado por la división de las tareas con la finalidad de distribuir la carga de trabajo en base a la especialidad de los integrantes.

Una vez llegados a este punto del proyecto, quedan las tres fases restantes: documentación, preparación del proyecto y montaje y exposición de la feria de proyectos.

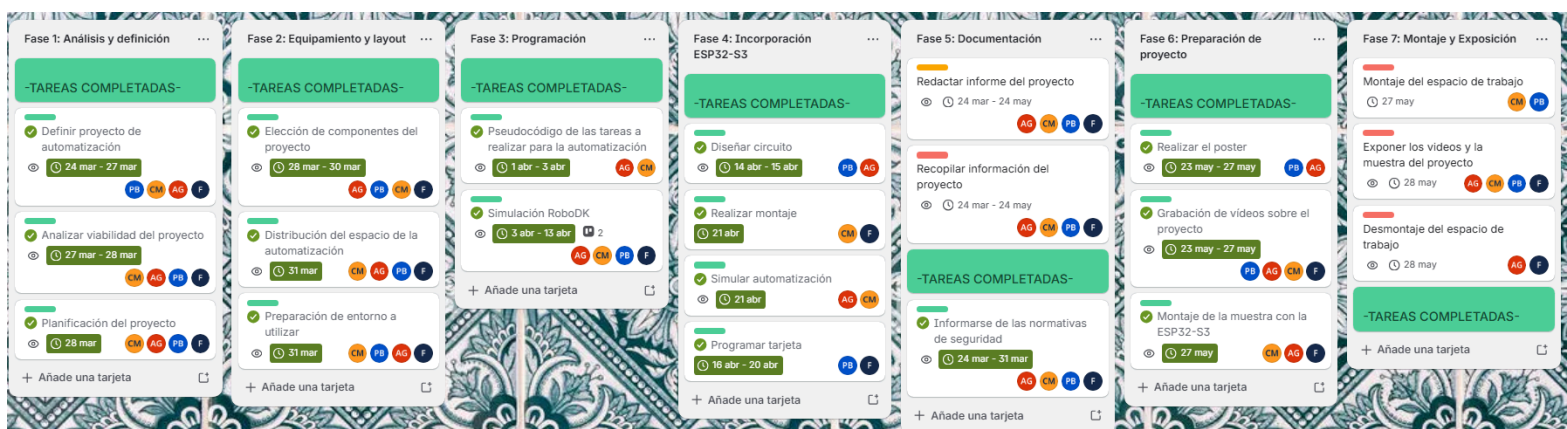
La fase de documentación se reduce principalmente a la recopilación de toda la información consultada y desarrollada durante las fases anteriores. Además, cabe tener en cuenta la aplicación de las normativas de seguridad -puesto que se han implementado unidades robóticas en el espacio de trabajo- en el proyecto cuya finalidad es garantizar la seguridad de los operarios en la planta. Al tratarse de la fase de recopilación de información, es importante que todos los miembros sean responsables de esta fase de manera equitativa, puesto que cada uno de los miembros ha realizado partes diferentes e imprescindibles para la realización exitosa del proceso automatizado. Es por ello, por lo que para alcanzar una mayor comprensión del proyecto desarrollado cada uno de los miembros debe plasmar todos los conocimientos adquiridos e implementados a lo largo del proyecto.



Finalmente, como una de las últimas fases se encuentra la fase de preparación del proyecto, la cual radica en la realización de la identidad corporativa y estrategia de difusión del proyecto orientado específicamente a la captación de socios estratégicos en el sector de la industria cerámica durante la última fase definida de proyecto, montaje y exposición de la feria de proyectos.



5.1.2 Listado de entregables del proyecto



Con el propósito de garantizar el cumplimiento estricto del cronograma, la trazabilidad de las tareas y un reparto equilibrado de la carga de trabajo entre los integrantes de la consultora externa, el equipo ha optado una metodología de gestión ágil mediante la plataforma Trello.

La captura de pantalla adjunta ilustra la estructura secuencial del proyecto, la cual se ha dividido en seis fases críticas. Cada columna representa un hito de entrega lógica que abarca desde el análisis de viabilidad inicial hasta la divulgación comercial de la solución de la feria de proyectos.

5.2.Pruebas y Validación

La estrategia de validación del proyecto se ha diseñado bajo un enfoque sistémico, dividiendo las pruebas en tres etapas progresivas que garantizan la robustez de la programación antes de su puesta en marcha física. En primer lugar, se contemplan las pruebas unitarias de código, enfocadas en verificar la estabilidad de los bucles infinitos en Python, la correcta conexión de los hilos de ejecución hacia la base de datos y la suscripción asíncrona a los *topics* de lectura del *broker* MQTT. En segundo lugar, se aplican pruebas de integración cinemática y lógica, las cuales evalúan la interacción en tiempo real entre los movimientos de los brazos robóticos UR5e y UR30, el avance condicionado de las cinco cintas transportadoras y la simulación de colisiones de las fotocélulas. Finalmente, se realizan pruebas de estrés de ciclo completo, donde se simula la producción continua de lotes heterogéneos de azulejos para analizar los tiempos de ciclo y la respuesta del sistema ante fallos concurrentes.

Como criterios de éxito indispensables para la aprobación de la simulación, se establecen los siguientes parámetros cuantitativos y cualitativos:

1. **Ausencia absoluta de colisiones y singularidades:** El detector dinámico de RoboDK debe permanecer en estado conforme (verde) durante el transcurso de todos los ciclos, garantizando trayectorias limpias.
2. **Trazabilidad íntegra de la información:** El 100% de las piezas procesadas deben registrar de forma síncrona en la base de datos relacional su número de serie formateado, estado de calidad y lote correspondiente sin provocar pérdidas de conexión o bloqueos en el cursor.
3. **Consistencia de la cola de datos:** La variable global *ColaAzulejos* debe indexar, actualizar y vaciar de forma secuencial y exacta los estados introducidos, evitando desbordamientos de memoria o detenciones del procesador mediante el sistema de esperas controladas de 0.1 segundos.
4. **Precisión volumétrica en el paletizado:** Las transformaciones matriciales tridimensionales de los robots deben posicionar las cajas y azulejos respetando estrictamente los incrementos espaciales calculados, consolidando un mosaico perfecto sin solapamientos geométricos.

5.2.1 Descripción de la simulación del proyecto de automatización (RoboDK)

La célula automatizada se ha modelado e implementado de forma íntegra en el software RoboDK, configurando un entorno de simulación que reproduce fielmente el layout industrial de una planta de cerámica. La estación virtual está compuesta por un robot colaborativo UR5e dedicado a las tareas de clasificación y un robot UR30 de alta capacidad destinado a la paletizado final. En el plano de herramientas (*tools*), el UR5e incorpora una garra de vacío SMC ZXP7A01-ZP20U-X1, mientras que el UR30 está equipado con un herramienta eléctrica OnRobot VGP20, ambos configurados con sus respectivos puntos de centro de herramienta (TCP) y cinemática inversa activa.

El flujo de materiales se compone de cinco cintas transportadoras sincronizadas a través de variables de entorno y supervisadas por cinco sensores láser fotoeléctricos SICK WL4S virtuales. El sistema de visión artificial Matrox Iris GTR se emula de manera lógica mediante scripts asociados a variables globales de estado. Para estructurar el espacio bidimensional y tridimensional, la estación cuenta con una jerarquía de marcos de referencia (*frames*) dinámicos vinculados a la base general de la célula, destacando las referencias móviles para las cajas de acumulación (CintaCaja1 y CintaCaja2) y los planos coordinados para la paletizado (Frame Pallet 1 y Frame Pallet 2). Todo el entorno virtual está vinculado de forma bidireccional mediante sockets TCP/IP con el microcontrolador físico ESP32-S3, logrando que los sensores y los actuadores de la simulación interactúen con un panel de operador real.

5.2.2 Procedimientos de validación y resultados esperados

Para validar de forma empírica la programación, se definieron y ejecutaron procedimientos de control específicos basados en las estructuras lógicas de los scripts de Python:

- **Validación Cinematica y Bucles de Espera en Cintas:** El procedimiento consistió en lanzar el avance de las líneas de transporte y forzar la colisión de un objeto con las fotocélulas de barrera. El resultado esperado, validado por software, confirmó que el método *Collision()* detiene el movimiento de la cinta de forma inmediata y que el sistema entra en un bucle de espera pasiva, reanudando la marcha de forma fluida únicamente cuando el objeto despeja el sensor, lo que confirma que las cintas no sufren bloqueos lógicos.
- **Validación de la Lógica de Clasificación de Calidad (UR5e):** Se introdujeron de forma síncrona valores variables en el parámetro *ColaAzulejos* empleando el panel de botones físicos vía MQTT. El resultado esperado fue la correcta lectura e interpretación de los estados lógicos mediante la estructura *match tipo_actual*: ante el Tipo 3 (azulejo roto), el robot permaneció en reposo seguro mientras la cinta evacuaba la pieza defectuosa hacia la papelera para su borrado físico mediante el método *.Delete()*. Ante los Tipos 1 y 2, el robot ejecutó con precisión la rutina de sujeción neumática estática (*setParentStatic*) y la transferencia hacia las cintas de acumulación 1 o 2 respectivamente, actualizando los registros en PostgreSQL mediante la librería *psycopg*.
- **Validación del Desfase Matricial de Posicionamiento (Paletizado UR30):** El procedimiento consistió en completar ciclos sucesivos de empaquetado para activar las señales de presencia *SenyalSensor3* y *SenyalSensor4*, obligando al UR30 a retirar las cajas llenas y depositarlas en los palets de descarga. El resultado esperado demostró la exactitud de la ecuación algebraica de traslación tridimensional implementada en el código. El robot aplicó de manera dinámica los incrementos de distancia en los ejes horizontales y verticales en base a los contadores de bucle (*x, y, z*), depositando cada caja con una separación milimétrica exacta y activando de forma segura las macros de señalización de palet lleno (*Palet1ON / Palet2ON*) al alcanzarse el volumen máximo de la matriz de paletizado.

5.3.Costes y Beneficios

5.3.1 Presupuesto detallado

El análisis financiero de la propuesta de automatización para la línea de producción de la planta de cerámica realizada por la consultora externa ha sido estructurado de forma modular, haciendo distinción entre la sección de elementos que componen la planta, servicios de ingeniería de integración y las inversiones en capital humano.

Primeramente, la parte principal del presupuesto consiste en la adquisición de bienes de equipo y suministros de hardware. Esta sección detalla el presupuesto requerido para la implementación de todos los elementos requeridos para la automatización. Así pues, todos los componentes han sido seleccionados bajo criterios industriales, buscando garantizar una buena compatibilidad con las APIs y además, flexibilidad funcional mediante protocolos.

PRESUPUESTO DEL PROYECTO	PRECIO	CANTIDAD
UR30	51.495,00 €	1
UR5e	27.557,00 €	1
Garra SMC ZWP7A01-ZP2OU-X1 Vacuum Gripper	2.110,26 €	1
Sensor SICK WL4S	189,55 €	5
Garra OnRobot VGP20 Vacuum gripper	7.657,81 €	1
Cámara Matrox Iris GTR	2.500,00 €	1
Cinta mecánica	4.906,55 €	5
Palé	13,51 €	2
TOTAL COSTES	96.429,68 €	116.827,59 €

Esta sección tiene en cuenta todos los componentes que conforman la celda, desde los brazos robóticos, herramientas de trabajo y periféricos de visionado, transporte y sincronización. Para obtener el valor total del presupuesto se han utilizado precios aproximados al mercado de cada uno de los elementos elegidos para la automatización. Como se puede observar, la tabla consta de dos celdas las cuales corresponden al total de costes; la celda situada más a la derecha es la que corresponde a la suma total de todos los elementos de la planta de trabajo. Se ha implementado esta estructura porque hay elementos, como son las cintas o los sensores que requieren de múltiples unidades. El sumatorio total de la sección de suministros de hardware y componentes del equipo asciende a un total de 116.827,59€.

La siguiente sección a tener en cuenta es la sección especializada en los gastos de ingeniería, instalación y configuración del proceso automatizado.

INSTALACIÓN		
Ingeniería y programación (Robo DK)	80,00 €	200 hora/s
Montaje mecánico y cableado	8.500,00 €	
Cuadro eléctrico y seguridad	4.500,00 €	
Transporte y puesta en marcha	2.500,00 €	
Instalación del UR30	8.500,00 €	
Instalación del UR5e	10.500,00 €	
TOTAL COSTES		50.500,00 €

La viabilidad del gemelo digital desarrollado no depende únicamente de los elementos físicos, sino que además se debe tener en cuenta para el presupuesto general del proyecto aspectos como las horas de ingeniería, la instalación de los equipos y la sincronización entre los componentes para lograr una planta automatizada y coordinada a la perfección.

En cuanto a los costes de ingeniería de software y programación se presupuesta alrededor de 200 horas de ingeniería dedicadas al desarrollo de la automatización, incluyendo la simulación y el desarrollo de las APIs. Para calcular el coste total al que asciende este aspecto se ha aproximado que la tarifa de programador o desarrollador de software se encuentra bajo una media de 80,00€. De esa forma se puede aproximar que el coste total orbita en torno a unos 16.000,00€. Luego, para el montaje e instalación de la parte mecánica, UR30 y UR5e principalmente, conlleva un coste de 27.500,00€. Previo a continuar con el resto de aspectos que abarca esta sección, cabe aclarar el motivo por el que la unidad robótica UR5e conlleva un gasto superior a la instalación del cobot UR30.

La unidad robótica UR5e requiere coordinarse con tres cintas mecánicas y una cámara de visionado artificial para la clasificación del azulejo. Asimismo, este robot se sitúa en el primer proceso de la automatización, la cual obliga a implementar un mayor grado de precisión en comparación con el trabajo que realiza el UR30, puesto que el UR5e debe colocar con delicadeza y precisión el azulejo en su caja y posición correspondiente. Tratando durante todo el proceso de no dañar el producto que está siendo empaquetado.

Para continuar con la sección de gastos de ingeniería, instalación y mantenimiento, se ha tenido en cuenta que para la parte física del proyecto, es decir, el ámbito de montaje y cableado es imprescindible no contar con los anclajes que se deben implantar en las bases de las unidades robóticas junto a la alineación y montaje de las cintas mecánicas con sus respectivos sensores de presencia. Asimismo, también abarca la gestión del cableado.

Por último respecto a esta sección es importante definir que aunque la implementación de unidades robóticas colaborativas elimina la necesidad de instalar medidas de seguridad como el vallado, sí se requiere de la instalación de controladores o comúnmente denominados PLCs, material como relés o fuentes de alimentación y cableado de seguridad que garantice la seguridad de los operarios en la planta.

Todos estos aspectos implican un aumento de 50.500,00€ en el presupuesto de la automatización.

Por último, la última sección que se ha tenido en cuenta para la elaboración del presupuesto detallado del proyecto automatizado se trata del mantenimiento de los equipos hardware instalados en planta.

MANTENIMIENTO ANUAL			
Mantenimiento del UR30		1.500,00 €	
Mantenimiento del UR5e		1.200,00 €	
Recambio de las ventosas		500,00 €	
Recambio de las correas de las cintas		270,00 €	
Consumo eléctrico		2.500,00 €	16 h/día
TOTAL COSTES			5.970,00 €

Cada componente de la planta de trabajo requiere de un mantenimiento anual para verificar y ajustar aquellos parámetros que con el paso del tiempo se hayan visto afectados. También, al haber elementos eléctricos se debe tener en cuenta el consumo eléctrico que conllevan estos elementos. Por consiguiente, se ha aproximado que el consumo eléctrico de la fábrica consta de alrededor de 16 horas, dividido en 8 horas en el turno matutino y el resto en el turno vespertino. La suma de todos los aspectos que conciernen al mantenimiento, junto al consumo eléctrico de éstos asciende a un total de 5.970,00€.

Al unificar el gasto total de todas las secciones del presupuesto, la inversión inicial que debe asumir la empresa cerámica es de un total de 173.297,59€. Al incluir de forma transparente los costes de capacitación, se asegura que la planta cerámica no adquiere tecnología de vanguardia, sino que cuenta con un equipo humano capacitado para operar con la máxima eficiencia y seguridad desde el primer día de producción.

5.3.2 Análisis Coste-Beneficio

Para evaluar la viabilidad de la inversión, se ha realizado un estudio de flujos de caja proyectado a un margen temporal de 5 años. Este análisis de retorno de inversión contrasta los gastos operativos de la implementación del proyecto junto con los retornos económicos que comienza a generar la misma.

RECUPERACIÓN DE LA INVERSIÓN						
Concepto	AÑO 0	AÑO 1	AÑO 2	AÑO 3	AÑO 4	AÑO 5
Inversión: Coste del proyecto	- 173.297,59 €	- €	- €	- €	- €	- €
Ahorros: salarios	- €	100.000,00 €	100.000,00 €	100.000,00 €	100.000,00 €	100.000,00 €
Gastos: Mantenimiento	- €	- 5.970,00 €	- 5.970,00 €	- 5.970,00 €	- 5.970,00 €	- 5.970,00 €
FLUJO ANUAL	- 173.297,59 €	94.030,00 €	94.030,00 €	94.030,00 €	94.030,00 €	94.030,00 €
TOTAL ACUMULADO	- 173.297,59 €	- 79.267,59 €	14.762,41 €	108.792,41 €	202.822,41 €	296.852,41 €

Como se puede observar, en el Año 0 no se contemplan ni gastos en mantenimiento ni ahorro en salarios. Esto ocurre porque el Año 0 se define en la industria como periodo base del proyecto. Esta primera etapa del proyecto solamente tiene como función representar la inversión neta necesaria para llevar a cabo la instalación de la automatización.

Sin embargo, a partir del Año 1 sí se contemplan los aspectos de ahorros salariales y gastos de mantenimiento de la infraestructura. Previo a indagar de forma más exhaustiva en la cifra total de los diferentes aspectos que se han tenido en cuenta para realizar el flujo de caja, se ha querido hacer un breve comentario sobre la evolución del flujo de caja del proyecto a lo largo de los años.

Una vez se ha superado el periodo de inversión inicial -Año 0-, el comportamiento de los flujos de cajas acumulados muestran una recuperación progresiva de la inversión realizada. Primeramente, en el Año 1 se puede visualizar como la deuda acumulada de la compañía disminuye drásticamente, pasando de una cifra elevada como se trata de

-173.297,59€ iniciales a -79.267,59€ finales. De igual manera, cabe prestar más atención en el Año 2 del flujo de cajas, puesto que es en este año cuando se constituye el hito financiero más crítico del proyecto. Esto sucede al sumar un segundo flujo de caja neto de un total de 94.030,00€, cuya procedencia se explicará más detalladamente a continuación, y se obtiene un total acumulado de +14.762,41€. Este cambio de signo implica que el proyecto se ha amortizado por completo y la empresa que ha implementado la automatización comienza a ganar beneficios. Por consiguiente, el comportamiento del flujo de caja los Años 3 y 4 son idénticos al Año 2, el flujo de caja sigue generando un beneficio acumulado que crece de manera exponencial. Por último, en cuanto al 5 Año, éste marca el límite impuesto por la empresa para el estudio de la viabilidad, por lo que se puede afirmar que el capital acumulado por la empresa asciende a 296.852,41€ libre de deudas. Esta cifra demuestra que la automatización de los procesos de empaquetado, paletizado y control de calidad no solo se trata de una solución eficiente, sino también con una gran rentabilidad financiera para la empresa.

Para una mayor comprensión de los datos del flujo de caja, a continuación se va a explicar el origen de cada apartado; inversión, ahorros en salarios y gastos en mantenimiento.

El valor tanto del apartado de inversión como el valor del apartado en gastos, equivale al coste del proyecto y al mantenimiento anteriormente explicado y calculado.

En torno al valor resultante de 100.00€ en ahorros en salarios anuales es fruto del cálculo del coste laboral total de los operarios que ceden sus tareas laborales repetitivas a las unidades robóticas y, serán reubicados en la realización de otras funciones de la planta de trabajo. Para el cálculo se ha considerado la existencia de dos turnos diarios -mañana y tarde- con una jornada laboral de 8 horas en total. Asimismo, se estiman 220 días laborables al año. Seguidamente, para el control de calidad y empaquetado se requiere a un operario por turno que sea el encargado de la inspección visual de los azulejos; además de depositar el producto cerámico en el interior de las cajas de empaquetado correspondientes. Mientras que en el puesto final de la línea de trabajo, el paletizado de las cajas requiere mínimo el trabajo y fuerza física de un operario que realice las tareas de transportar de forma manual las cajas llenas de azulejos al final de la cinta de empaquetado hasta la posición correspondiente del palé. Es por ello, por lo que se puede concluir el ahorro de mínimo 2 operarios por turno, es decir, en total 4 operarios. Otro aspecto que se ha considerado para el cálculo de ahorros en salarios es el sueldo anual bruto de un operario, se ha aproximado que alcanza alrededor de 25.000€.

$$\text{Total ahorro en salarios: } 4 \times 25.00\text{€} = 100.000\text{€}$$

Una vez ya se ha determinado el origen del ahorro en salarios, para obtener un flujo de cajas completo se debe tener en cuenta el flujo neto anual de la empresa. El flujo anal es el conjunto de capital real que entra en la empresa cada año de manera limpia. Para obtener el valor de este parámetro, se realiza la siguiente operación:

$$\text{Flujo Neto Anual} = 1000.000\text{€ (Ahorro en salarios)} - 5.970\text{€ (Gastos de mantenimiento)} = 94.030\text{€}$$

La cifra total de 94.030€ se mantiene constante desde el Año 1 hasta el Año 5, ya que se trata de la implementación de un escenario de producción estable.

5.3.3 ROI

Como instancia final del análisis económico del proyecto se han calculado los indicadores económicos de viabilidad con el fin de determinar el plazo de amortización y el índice de rendimiento del capital invertido.

En primer lugar, en relación con el parámetro *Payback* o también denominado plazo de recuperación, es el tiempo extracto que tarda una inversión en recuperar su capital inicial a través de los ingresos netos generados. Se trata de un indicador de riesgo y liquidez; cuanto más corto sea el plazo, menor es el riesgo y más líquida es la inversión. En el sector de la industria, para afirmar que un proyecto es rentable, el *PayBack* debe ser inferior a los 2 años.

En torno al proyecto, el análisis del periodo de retorno determina que la inversión inicial de -173.297,59€ se recuperará en su totalidad en un plazo de 1.84 años -aproximadamente 1 año y 10 meses-. En efecto, el proyecto de automatización al encontrarse bajo la frontera de los 2 años se puede afirmar que se trata de una inversión de alta prioridad estratégica para la empresa. A partir del Año 2 la inversión inicial se amortiza y comienza a obtener un flujo de caja positivo, permitiendo a la empresa abordar otras necesidades de la planta de trabajo.

En suma, se ha realizado de igual manera el estudio del retorno de la inversión (ROI) para profundizar en el estudio de la viabilidad del proyecto. Este parámetro mide el resultado económico generado de las inversiones realizadas. Es por ello por lo que su cálculo es fundamental para determinar la rentabilidad de los proyectos empresariales.

El retorno de la inversión se calcula partiendo de la siguiente fórmula:

$$\text{ROI} = (\text{Ingresos} - \text{Inversión}) / \text{Inversión} = (94.303,00 \text{ €} - 173297,59\text{€}) / 173297,59\text{€} = 171,30\%$$

La cifra alcanzada certifica que el beneficio neto generado triplica el coste del CAPEX inicial. Desde la perspectiva de la gestión de riesgos este porcentaje proporciona un colchón financiero sumamente amplio para la empresa. Incluso si surgieran imprevistos como paradas de producción o roturas en las unidades robóticas, el proyecto seguiría siendo rentable, ya que el margen de beneficio es muy superior al coste.

6. Normativa y regulación. Seguridad

6.1. Impacto en los puestos de trabajo

Al implantar la celda automatizada, la forma de trabajar cambia por completo. No obstante, la idea no es destruir empleo ni echar a nadie, sino recolocar a los trabajadores en puestos mucho más seguros y cualificados.

- **Eliminando la tareas más duras:** los 4 operarios que antes pasaban su turno revisando los azulejos a simple vista y cargando a pulso las cajas pesadas para armar los palés, se retiran de esa zona tan físicamente dura evitando bajas por dolores de espalda o cortes con los azulejos rotos.
- **Nuevos roles en la planta:** a estos mismos trabajadores se les dará una formación básica para que pasen a ser los supervisores de la celda robótica. Sus nuevas tareas serán controlar el buen funcionamiento observando el gemelo digital en el ordenador, revisar las alertas de los LEDs, la supervisión del buen funcionamiento de la cámara y coordinar cuando el pallet necesita ser retirado.
- **Ahorro económico:** al reorganizar la plantilla y optimizar los turnos, la empresa consigue un ahorro de unos 100.000€ al año en costes de mano de obra directa. Esto es clave ya que permite amortizar lo que cuesta montar la celda en solo 1 año y 10 meses, mejorando considerablemente las condiciones laborales y la salud de la plantilla.

6.2. Consideraciones de seguridad

El diseño de la celda de trabajo se ha basado en la tecnología de robots colaborativos, lo que elimina la necesidad de instalar jaulas o vallados físicos tradicionales. Esto se traduce en una mayor eficiencia de espacio y una mejor integración en el entorno de la nave.

La seguridad se gestiona a través de la propia naturaleza colaborativa de los brazos robóticos, los cuales integran sensores de fuerza y esfuerzo que detectan cualquier contacto imprevisto, deteniendo el movimiento de forma inmediata. Esta capacidad se complementa con una arquitectura de control de seguridad que incluye paradas de emergencia cableadas y redundantes, asegurando que cualquier intervención humana en el área de trabajo ocurra en condiciones controladas y seguras.

6.3. Cumplimiento de regulaciones y estándares

La propuesta de automatización se ha diseñado bajo el estricto cumplimiento de la normativa europea y nacional vigente en el ámbito de seguridad industrial y maquinaria. Las normativas aplicadas son:

- **Directiva de Màquines 2006/42/CE:** Es la norma fundamental que regula la seguridad de los sistemas integrados en la celda de trabajo. Dado que se utilizan robots colaborativos (cobots), se ha realizado una Evaluación de Riesgos específica de sus respectivas aplicaciones. Aunque los cobots permiten prescindir de vallados, la normativa exige que el riesgo de impacto se mantenga dentro de los límites de fuerza y presión definidos en la norma **ISO/TS 15066**.
- **Especificaciones Técnicas ISO/TS 15066 (Robots Colaborativos):** Esta normativa exige que los riesgos de impacto físico se mantengan dentro de los límites de fuerza y presión tolerables en el caso de impacto con el operario. Como consecuencia, se implementa la validación mediante sensores de fuerza internos y *Limitación de Potencia y Fuerza (PFL)* en los controladores de ambas unidades robóticas. Asimismo, para aumentar la seguridad en la celda es recomendable la instalación de sensores como el sensor implementado en las cintas mecánicas, en el suelo. De esta manera, si un operario se aproxima a la zona de trabajo de los cobots y el sensor lo detecta esto envía una señal a ambas unidades robóticas que permite reducir sus movimientos y reduciendo a una velocidad segura.
- **Distancias Mínimas (ISO 10218-2:2011 y UNE-EN ISO 13854):** De acuerdo con la distribución del *layout*, la base del UR5e se sitúa a 200 mm de las cintas circundantes y la del UR30 a 950 mm. Para evitar puntos de aplastamiento donde un operario pueda quedar atrapado entre la base del robot y las estructuras fijas de la planta, la normativa exige una distancia libre mínima de 500 mm. Al no disponer de este espacio físico en el primer proceso, se configuran de forma obligatoria límites de restricción de espacio en el software de control de los robots, bloqueando cinemáticamente cualquier trayectoria que tienda a invadir o presionar contra zonas muertas que puedan generar atrapamientos.
- **Seguridad Eléctrica (UNE-EN 60204-1):** Toda la infraestructura, incluyendo el controlador ESP32-S3 y la interconexión con las cintas transportadoras, cumple con las normas de seguridad eléctrica para maquinaria, garantizando la protección contra sobrecargas y garantizando la puesta a tierra.
- **Compatibilidad Electromagnética (CEM):** Se han seguido las guías de diseño para asegurar que el sistema de visión (Matrix Iris GTR) y las comunicaciones (MQTT) no sufran interferencias, cumpliendo con la directiva 2014/30/UE.

6.4. Documentación y material entregable al cliente

Para asegurar el mantenimiento a largo plazo, se entregaría a la empresa el siguiente informe :

1. **Manual de Operación y Mantenimiento:** Un documento que detalla el arranque / parada del sistema, los procedimientos de limpieza y la guía de resolución de problemas básicos (*troubleshooting*) para los nuevos supervisores de celda.
2. **Dossier Técnico de Seguridad:** Incluye la Declaración CE de Conformidad, el marcado CE de la celda y los certificados de seguridad de los cobots UR5e y UR30, esenciales para futuras auditorías de riesgos laborales.
3. **Gemelo Digital y Modelo Robótico:** Se entregará el proyecto desarrollado en RoboDK, incluyendo todas las cinemáticas y trayectorias, para que el equipo de ingeniería pueda realizar simulaciones de nuevos formatos de forma virtual antes de aplicarlos en la planta real.
4. **Repositorio de Software (Código Fuente):** Acceso a los scripts en Python desarrollados para el control de los cobots y la lógica de visión, debidamente documentados para permitir futuras actualizaciones o cambios de formato por parte del equipo técnico interno.
5. **Plan de Formación:** Un conjunto de fichas técnicas para la capacitación del personal de planta en sus nuevos roles de supervisión, incluyendo el manejo de las interfaces de control y monitorización de alertas.

7. Desarrollo de Software y Algoritmos

7.1. Listado tecnológico

Este apartado detalla tanto los componentes físicos y virtuales (hardware) que integran la célula de automatización, como las herramientas de software y lenguajes de programación (tecnologías de implementación) utilizados para el control, la simulación y la comunicación del sistema.

7.1.1 Componentes del Sistema (Hardware e Infraestructura Virtual)

La solución propuesta se divide en dos entornos interconectados: el entorno de simulación robótica (robótica industrial y transporte) y el entorno de control y comunicación local (electrónica y señalización).

Categoría	Componente / Modelo	Cantidad	Función Principal en el Proyecto
Robótica	Robot Colaborativo UR5e (Universal Robots)	1	Encargado de la manipulación inicial de azulejos y posicionamiento en la estación de control de calidad.
Robótica	Robot Colaborativo UR30 (Universal Robots)	1	Robot de alta carga encargado del paletizado final de las cajas empaquetadas.
Garras (EOAT)	SMC ZXP7A01-ZP20U-X1	1	Garra de vacío por ventosas acoplada al UR5e para la sujeción segura de los azulejos.
Garras (EOAT)	OnRobot VGP20	1	Garra de vacío eléctrica de alta capacidad acoplada al UR30 para soportar el peso en el paletizado.
Transporte	Cintas transportadoras genéricas	5	Guiado y traslado de los azulejos y las cajas llenas entre las diferentes etapas del proceso
Visión Artificial	Cámara Matrox Iris GTR	1	Cámara inteligente encargada de la inspección óptica para la detección de defectos en los azulejos.
Sensórica	SICK WL4S Laser Sensor	5	Sensores fotoeléctricos de barrera para detectar la presencia y posición de los azulejos y las cajas en las cintas.
Control / IoT	Microcontrolador ESP32-S3	1	Unidad central de comunicación Wi-Fi encargada de gestionar las señales del entorno físico.
Electrónica	Leds de señalización	2	Indicadores visuales del estado del sistema y del paletizado.

Electrónica	Pulsadores	5	Interfaz física para comandos manuales.
Componentes	Resistencias y Cableado	7/-	Componentes pasivos para la protección de circuitos de los LEDs y conexiones de la ESP32.

7.1.2. Tecnologías de Implementación (Software y Entornos de Desarrollo)

Para la integración, control y comunicación de todos los elementos de la célula, se ha desarrollado una arquitectura de software multitarea que combina simulación robótica, programación lógica, sistemas embebidos y gestión de datos.

El núcleo del entorno virtual se gestiona a través de la interfaz de RoboDK, plataforma utilizada para la simulación 3D y la programación offline (OLP) de las estaciones robóticas. Este software permite coordinar el layout tridimensional, validar la cinemática de los robots UR5e y UR30 y garantizar trayectorias fluidas y libres de colisiones. La lógica interna de esta simulación se dinamiza mediante scripts en Python, lenguaje que actúa como cerebro integrador del proceso. Python se encarga de automatizar las tareas complejas dentro de RoboDK, tales como el apilado y paletizado completo de azulejos y cajas, el funcionamiento de las cintas, la simulación de colisión con los sensores de barrera y el recibo y envío de mensajes de MQTT.

Por otro lado, la interacción con el entorno físico y el control de los operadores se implementa en el ecosistema Arduino (C/C++), entorno empleado para programar el firmware del microcontrolador ESP32-S3. Este código se encarga de realizar la lectura en tiempo real de los cinco pulsadores analógicos, gestionar el encendido de los LEDs de estado y empaquetar dicha información. Acto seguido, los datos son transmitidos de forma inalámbrica hacia la simulación mediante el protocolo de comunicación Wi-Fi (TCP/IP), permitiendo que las acciones físicas sobre el panel de botones alteren el comportamiento de la célula virtual de forma inmediata.

Finalmente, el sistema incorpora una capa de supervisión y analítica basada en SQL (Structured Query Language). Mediante esta tecnología, el script de control interactúa con una base de datos relacional para registrar la información de producción en tiempo real. Esto permite almacenar la trazabilidad completa del proceso, incluyendo el conteo de azulejos procesados, la clasificación de piezas conformes frente a las defectuosas o las rotas, los pedidos recibidos o los clientes de la empresa, dotando al proyecto de una infraestructura sólida alineada con los estándares de la Industria 4.0.

7.2. Descripción de la implementación

7.2.1 Estructura de los programas, división en componentes, tareas y funciones principales

El desarrollo software del proyecto se ha dividido en dos grandes bloques: el firmware del sistema embebido, encargado de la interacción física y las comunicaciones de campo, y la aplicación de control central en RoboDK con Python.

Por parte del sistema embebido, el código se ha diseñado bajo una estructura modular estricta. Esta división en múltiples ficheros o componentes facilita el mantenimiento, la escalabilidad y la depuración del sistema. La estructura del firmware se divide en tres áreas funcionales principales.

Primeramente, para la configuración global y la gestión de utilidades, se ha implementado “config.h” que actúa como núcleo del proyecto. Contiene todas las definiciones de macros, credenciales de red (SSID/Password), parámetros de conexión al broker (broker.emqx.io), mapeo de pines físicos (GPIOs) y la definición estática de la jerarquía de topics MQTT. En cuanto al seguimiento de trazas, “logger.ino” implementa un sistema o *logging* personalizado. Clasifica los mensajes de salida por consola serie según su nivel de severidad (TRACE, DEBUG, INFO, WARN, ERROR, FATAL). Esto permite ajustar el nivel de verbosidad durante la depuración sin necesidad de eliminar instrucciones Serial.print a lo largo del código. Además, “funciones.ino” agrupa subrutinas auxiliares de uso transversal, como el control del LED integrado de la placa para mostrar el estado operativo.

A continuación, con lo que respecta a las comunicaciones, “wifi_lib.ino” encapsula toda la lógica de conectividad inalámbrica. Gestiona la conexión inicial (wifi_connect) y el mantenimiento del enlace (wifi_loop y wifi_reconnect), asegurando que el microcontrolador intenta reconectarse automáticamente si se pierde la señal, con soporte preparado para conexiones seguras (SSL). Además, “mqtt_lib.ino” es un componente crítico que administra la pasarela de mensajería. Inicializa el cliente PubSubClient, gestiona las suscripciones, y publica mensajes. También, aloja la función callback (mqttCallback) que intercepta cualquier mensaje entrante, aplicando aquí el filtro de deduplicación antes de enrutar la orden hacia la lógica de control.

Finalmente, en lo que respecta al sistema embebido, se encuentra la lógica de control e interacción de hardware (I/O). Para ello, “setup.ino” inicializa la configuración de los modos de los pines (INPUT_PULLUP para la botonera y OUTPUT para los actuadores) y establece los estados lógicos seguros de arranque. De esta manera, “buttons.h” y “comunicaciones_buttons.ino” administran las señales de entrada. Mediante la estructura de datos BotonMQTT, este módulo iterativo lee el estado de los pulsadores físicos aplicando un control de temporización (antirrebote de 50 ms) y, al detectar un cambio de estado válido, ordena la publicación del payload correspondiente. Y para gestionar las señales de salida,

“*comunicaciones_led.ino*” recibe las directrices procesadas por el callback de MQTT y traduce los comandos textuales (“on”, “off”) en señales eléctricas de nivel alto o bajo para conmutar los LEDs indicadores de estado de los palets y del sistema.

La ejecución principal del microcontrolador sigue un flujo no bloqueante: tras inicializar el hardware y las redes, el bucle principal (loop) invoca de manera continua las rutinas de mantenimiento de Wi-Fi y MQTT, seguido de la función *on_loop()* que evalúa constantemente el estado de los periféricos. Esta arquitectura reactiva asegura que la placa procese eventos de entrada y reciba comandos de salida de forma simultánea e ininterrumpida.

Por otra parte, dentro del entorno de desarrollo de RoboDK, la arquitectura de control se ha estructurado en dos tipologías de programas claramente diferenciadas: los scripts avanzados de programación en Python y las secuencias nativas integradas a través de la interfaz gráfica del software. Mientras que los primeros asumen la lógica matemática y de control de los dispositivos, los programas nativos de la interfaz de RoboDK se han implementado como macros de supervisión y gestión del entorno, orientados a simplificar la operatividad, el mantenimiento y la preparación de la simulación.

Entre estas secuencias integradas destaca en primer lugar el denominado “PROGRAMA FINAL”, el cual actúa como el secuenciador principal del proyecto y se encarga de inicializar y llamar de manera ordenada a todos los scripts de Python requeridos para que la simulación comience su ejecución cíclica de forma fluida, coordinada y exenta de conflictos de sincronización. A este le sigue el programa “RESET TOTAL”, una subrutina de mantenimiento diseñada específicamente para restaurar el estado inicial de la célula de trabajo al posicionar nuevamente todas las cintas transportadoras en su origen y eliminar de la simulación aquellos azulejos residuales o fuera de posición que no se encuentren clasificados en la línea de entrada principal. Finalmente, el sistema incorpora las funciones “Set a 0” y “Set a 0 bd”, las cuales actúan como una herramienta de seguridad y puesta a punto encargadas de restablecer el valor nulo o inicial en la totalidad de las variables globales y registros de entorno de RoboDK. Estas últimas macro permiten reiniciar el ciclo productivo completo desde cero ante cualquier parada imprevista o reconfiguración del sistema, garantizando que no se produzcan solapamientos de datos ni altercados en la lógica de control interna.

El desarrollo del entorno de simulación automatizada en RoboDK se complementa con una arquitectura de scripts en Python que actúan como la lógica de control distribuida del sistema. Estos programas se dividen de manera modular en tres bloques funcionales bien definidos: el control cinemático de las cintas transportadoras, la gestión de comunicaciones mediante el protocolo MQTT y los algoritmos lógicos que gobiernan las estaciones robóticas.

Los scripts que regulan el funcionamiento de las cinco cintas transportadoras operan bajo una directriz común basada en eventos de proximidad y contacto físico virtual. Tras realizar la importación de módulos y la inicialización de las variables y marcos de referencia (*frames*) de RoboDK, cada programa se introduce en un bucle principal infinito. Dentro de

este ciclo, se evalúa constantemente el estado de una variable booleana de detección. La cinta avanza de manera ininterrumpida mientras no se registre una interacción directa entre el objeto transportado y su fotocélula correspondiente, condición que se monitoriza mediante la función nativa *fotocelula.Collision(objeto)*.

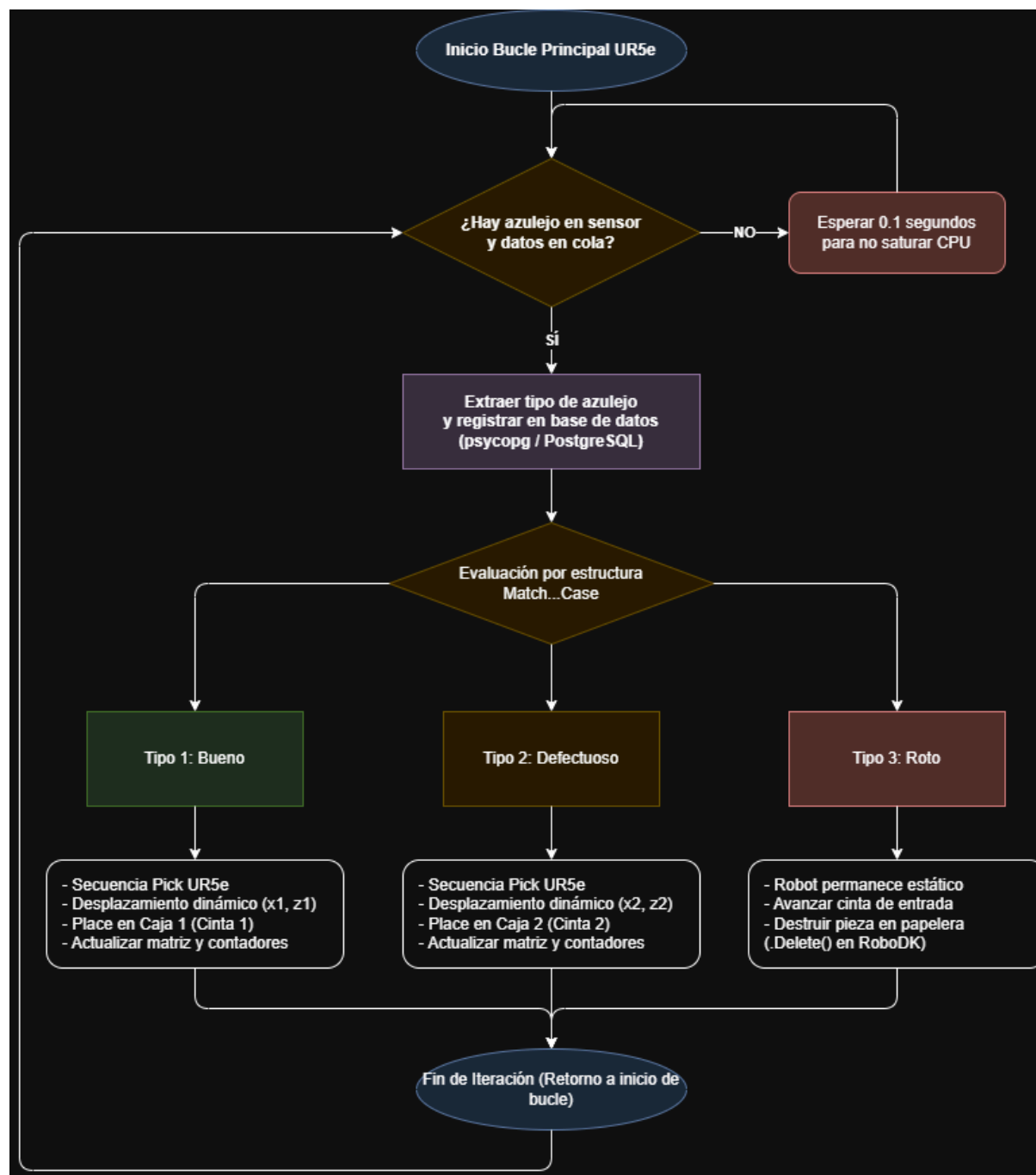
En el momento en que se valida dicha colisión, el software detiene el avance y activa un bucle de espera controlado que se mantiene activo hasta que el objeto despeja el campo de visión del sensor. Si bien este comportamiento es idéntico para la mayoría de las líneas de transporte, los scripts asociados a las cintas 1 y 2 integran subrutinas adicionales para garantizar la trazabilidad. Tras liberarse el bucle de espera en estas dos líneas, el programa interactúa con la base de datos relacional para registrar la salida de una nueva caja completa. Acto seguido, el contenedor se desplaza hasta situarse bajo el arco de empaquetado, donde se eliminan los azulejos de su interior para simular el proceso de sellado físico, reiniciando finalmente la posición de la cinta para comenzar un nuevo ciclo productivo.

La sincronización entre el entorno físico y la célula virtual se consolida a través de una red de mensajería estructurada bajo el protocolo MQTT, la cual interactúa directamente con el sistema embebido basado en el microcontrolador ESP32-S3. Los scripts encargados de esta infraestructura se dividen estrictamente según la dirección del flujo de datos:

- **Scripts de Recepción y Control:** Diseñados bajo una jerarquía segmentada de *topics*, programas como *LeerMQTTS* y *LeerMQTTB* se suscriben a los canales del *broker* para realizar lecturas asíncronas y continuas. Al recibir una carga útil (*payload*), capturan el mensaje y lo redirigen a los módulos de ejecución *RobotControllerS* y *RobotControllerB*. Específicamente, *RobotControllerS* intercepta el estado de los azulejos enviado desde los pulsadores físicos para emular las lecturas de una cámara de visión artificial real. Este valor se almacena en una cola de datos global denominada *ColaAzulejos*, encargada de indexar secuencialmente la calidad de las próximas piezas que ingresarán al empaquetado. Por su parte, *RobotControllerB* monitoriza las peticiones de los operadores para gestionar tareas de mantenimiento en la célula, tales como la retirada programada de palets completos y el apagado selectivo de los indicadores luminosos.
- **Scripts de Transmisión y Señalización:** En contraposición a los módulos de lectura, los programas que publican hacia el *broker* MQTT no se ejecutan en bucles infinitos, sino de manera imperativa ante eventos puntuales del proceso para optimizar los recursos del sistema. Estos módulos constan de parejas de scripts dedicados a enviar cadenas específicas de activación (ON) o desactivación (OFF). Su finalidad primordial es actualizar de forma remota los estados de los indicadores LED físicos, notificando de forma instantánea si la célula se encuentra en régimen de operación activo o si alguno de los dos palets de descarga ha alcanzado su capacidad máxima de almacenamiento.

El script que gobierna el brazo robótico colaborativo UR5e se encarga de discriminar la calidad del producto y realizar el primer nivel de empaquetado. El programa inicia con la

importación de las APIs *robodk.robolink* y *robodk.robomath*, el establecimiento de una conexión segura a la base de datos PostgreSQL mediante el adaptador *psycopg*, y el mapeo de todas las coordenadas de aproximación, paso y colocación (*targets*) asociadas a la herramienta de vacío SMC ZXP7A01.



Al ingresar en su bucle de control, el script extrae la cadena almacenada en el parámetro global *ColaAzulejos* e inspecciona el estado de la señal digital *SenyalSensorA*. Si no se detecta la presencia física de una pieza o si la cola se encuentra vacía, el hilo de ejecución se suspende durante 0.1 segundos para prevenir la saturación del procesador. Cuando ambas condiciones se validan, el script extrae el primer elemento de la lista y

actualiza de inmediato el parámetro de RoboDK; si la extracción vacía la cola, se asigna por defecto el valor de control unitario (1).

Antes de realizar cualquier movimiento físico, el programa formatea un número de serie único y ejecuta una sentencia SQL de inserción (INSERT INTO *azulejos.Azulejo*) a través de un cursor para guardar de manera persistente el identificador, el estado cualitativo de la pieza (clasificada como "bueno", "defectuoso" o "roto") y su lote correspondiente.

A partir de este punto, la trayectoria se bifurca según el tipo de azulejo procesado: Si el tipo de azulejo es igual a 3 (pieza rota), el robot permanece estático; se envía una orden cinemática a la cinta transportadora de entrada para avanzar un incremento lineal fijo, desplazando la pieza defectuosa directamente hacia el contenedor de basura, donde es eliminada de la simulación mediante el método *.Delete()*.

Por otro lado, si el tipo de azulejo es 1 o 2 (Piezas buenas / defectuosas), el UR5e ejecuta una interpolación conjunta (*MoveJ*) hacia el punto de paso seguro y desciende en interpolación lineal (*MoveL*) sobre el alimentador empleando el sistema de coordenadas de la cinta. Tras activar la sujeción estática de la ventosa sobre la primera geometría disponible (*setParentStatic*), el robot se eleva e inicia una estructura de selección *match...case*. Si la pieza es de Tipo 1, el robot aguarda la señal del sensor de la Caja 1, se traslada a la zona de empaquetado correspondiente y calcula dinámicamente un desfase de posición relativo en tres dimensiones empleando transformaciones matriciales homogéneas (*robomath.transl*) basadas en los índices incrementales de la caja (*x1*, *z1*). Una vez depositada la pieza y transferido su vínculo al contenedor de destino, se evalúan los límites de la matriz de empaquetado. Si el índice horizontal alcanza el límite de la fila, se incrementa la capa vertical y se resetea la coordenada base; al completarse la caja en su totalidad (límite *z1* = 5), se genera un pulso lógico de finalización (*Done1* = 1) para alertar al sistema de transferencia. Este procedimiento exacto se replica simétricamente bajo el case 2 empleando los registros de control (*x2*, *z2*) y las referencias espaciales de la segunda línea de transporte.

El último eslabón del proceso está comandado por el script del robot UR30, cuyo propósito exclusivo es la manipulación de alta carga para la consolidación de palets a partir de cajas terminadas. Tras enlazar los subprogramas de señalización externa (*Palet1ON*, *Palet2ON*) y definir las matrices de paso, reposo y sujeción de la herramienta OnRobot VGP20, el script se bloquea en un bucle de espera activa condicionado por las señales de presencia de los sensores finales de carrera de las cintas 3 y 4 (*SenyalSensor3* y *SenyalSensor4*).

Cuando una de las dos líneas de transporte reporta una caja llena, el script rompe la espera y asigna un identificador de tarea (tipo = 1 o tipo = 2), implementando por software una prioridad fija en favor de la cinta 4 en caso de colisiones temporales simultáneas. A través de un bloque *match...case*, el robot ejecuta la secuencia de paletizado:

Al activarse el escenario de la cinta 3 (Caja Tipo 1), y siempre que el palet homólogo no se encuentre bloqueado por una alarma de capacidad (*LuzPalet1* == 0), el UR30 abandona

su posición de reposo global y se desplaza linealmente hacia el extremo de la línea de transporte. El script lee el primer elemento hijo del contenedor virtual, acopla la caja firmemente al herramental de vacío y se desplaza hacia el plano de trabajo del Palet 1.

Para determinar las coordenadas exactas de descarga, el script recupera las variables de indexación espacial x_3 , y_3 , z_3 desde el entorno de RoboDK. Mediante el uso de álgebra computacional, se computa una matriz de traslación tridimensional tridimensional pura:

$$desplazamiento_1 = transl(x_3 \cdot 180, y_3 \cdot 280, z_3 \cdot (-120))$$

Esta matriz se multiplica directamente por la postura base del objetivo (*target_Place1.Pose()*), obteniendo una localización espacial única y adaptada para cada ciclo de apilado. El robot realiza la aproximación en alta velocidad, reduce sus parámetros cinemáticos a valores seguros de precisión (50mm/s de velocidad lineal y 20°/s de velocidad de junta) para el contacto, libera la caja sobre la estructura del palet y retorna al punto de tránsito.

Finalmente, el programa actualiza los contadores del mosaico tridimensional: el índice de fila x_3 se incrementa de forma unitaria hasta alcanzar el límite del palet ($x_3 = 3$), momento en el cual avanza la columna y_3 y se restablece el origen horizontal. Cuando la capa bidimensional se completa ($y_3 = 2$), se incrementa el eje de altura z_3 . Al completarse el volumen máximo estipulado (capa $z_3 = 3$), el script ejecuta de forma paralela el subprograma *progl.RunProgram()*, encendiendo el LED físico de aviso de palet lleno a través de MQTT y bloqueando futuras operaciones de carga en dicha estación hasta su recogida. Los nuevos valores de indexación se actualizan en las variables de entorno de RoboDK antes de retornar al brazo a su posición segura de reposo. Esta secuencia se repite con idéntica lógica matemática en el *case 2* para la gestión y paletizado del Palet 2 utilizando sus propias referencias espaciales y variables de control (x_4 , y_4 , z_4).

7.2.2 Algoritmos de programación avanzada y estructuras de datos de programación

En este apartado se detalla la implementación técnica y el funcionamiento lógico de los algoritmos y estructuras de datos integradas en el firmware del ESP32-S3. Como se estableció en la fase de diseño, se ha prescindido de soluciones excesivamente pesadas en favor de un enfoque de bajo coste computacional y alta eficiencia, adecuado para las limitaciones de memoria de un microcontrolador y estrictamente enfocado a la resolución de problemas reales en la planta automatizada.

En primer lugar, la lógica de duplicación de mensajes MQTT se implementa mediante la combinación de un algoritmo de hashing no criptográfico y un búfer circular. Específicamente, al interceptar un mensaje en la rutina de callback, el sistema concatena el topic y el payload entrantes empleando un carácter delimitador. Esta cadena resultante se

procesa mediante un algoritmo, el cual inicializa una variable entera de 32 bits y recorre secuencialmente cada carácter del mensaje. Para lograr la máxima eficiencia en el procesador, el algoritmo calcula el nuevo hash aplicando desplazamientos lógicos de bits a la izquierda y sumando el valor binario del carácter evaluado, evitando así multiplicaciones complejas de alto coste.

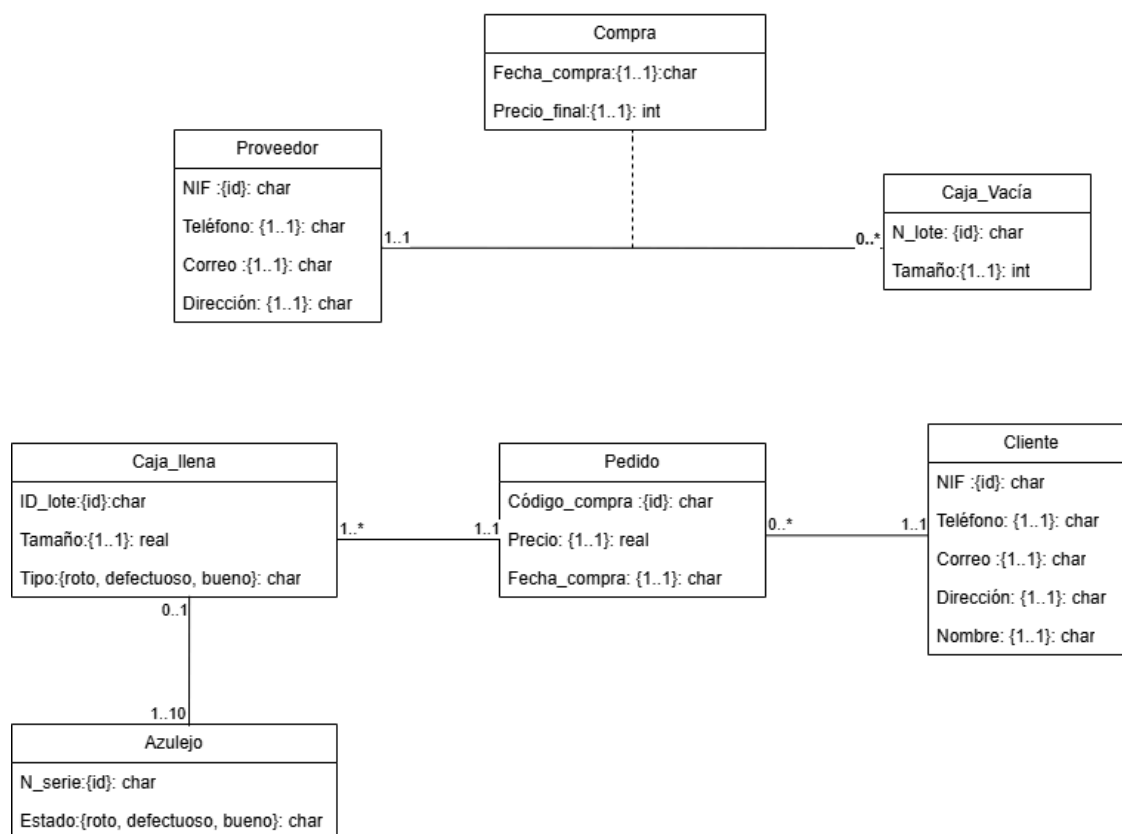
El valor numérico resultante se evalúa mediante una búsqueda lineal dentro de un arreglo estático de tamaño fijo ($N=32$), el cual actúa como histórico de los últimos comandos procesados. Si el hash ya figura en esta estructura, el mensaje se descarta automáticamente antes de que afecte a la lógica de la celda. En caso contrario, el nuevo hash se almacena en el índice actual del búfer. Para mantener la estructura operativa de forma indefinida, se emplea un índice dinámico que se incrementa en cada nueva inserción; al alcanzar el límite estático del arreglo, este puntero se reinicia a cero. Esta técnica sobrescribe de forma cíclica la entrada más antigua, garantizando una ocupación de memoria estrictamente acotada a $O(N)$ y una complejidad temporal predecible.

Por otro lado, respecto al tratamiento de múltiples componentes de hardware, se ha diseñado una estructura de datos compuesta con la finalidad de organizar de manera eficiente la gestión de los botones físicos y su correspondencia con los eventos MQTT. En el archivo de cabecera pertinente, se define la estructura *BotonMQTT*, cuya finalidad es agrupar en un bloque de memoria continua y lógica todos los atributos esenciales para cada actuador. Este registro encapsula los parámetros de configuración estáticos (el pin físico de entrada, el topic de publicación y el payload correspondiente) junto con las variables dinámicas de estado necesarias para el control de temporización, tales como la última lectura registrada, el estado lógico estable y la marca de tiempo en milisegundos.

Esta encapsulación de datos permite que el procesamiento de las señales se resuelva empleando un único arreglo estático que instancia todos los botones del sistema. Durante la ejecución continua, el firmware recorre este arreglo mediante un bucle iterativo uniforme, aplicando una lógica de control antirrebote calibrada a 50 milisegundos para cada elemento. Si la función detecta una transición válida y estable hacia el estado activo (LOW), extrae directamente de la estructura los punteros de texto necesarios para ejecutar la publicación MQTT. De esta forma, se evita duplicar código por cada pulsador físico implementado y se dota al sistema de una alta escalabilidad, puesto que la integración de futuros actuadores se limitará a la adición de nuevos elementos al arreglo, manteniendo intacta la lógica algorítmica principal.

7.3. Esquema de la BD utilizada

En este apartado se detalla la implementación física de la base de datos, creada en PostgreSQL bajo un esquema propio. Este modelo lógico-relacional está formado por varias tablas interconectadas que permiten hacer un seguimiento completo del producto y asegurar que los datos generados por la celda automatizada se guardan correctamente y sin errores.



La parte principal de la base de datos se centra en las tablas que registran la producción física. La tabla *Azulejo* es la unidad básica, donde cada pieza se identifica con un número de serie único que funciona como clave primaria. Para evitar errores en los datos que envía la unidad de control, se ha añadido una restricción tipo *CHECK* en la columna de estado, limitando los valores permitidos a "bueno", "defectuoso" o "roto". Además, cada azulejo se asocia a una caja mediante una clave ajena que apunta a la tabla *Caja_llena*. Esta tabla, que usa el identificador de lote como clave principal, también incluye la restricción *CHECK* para el tipo de piezas que contiene, anota el tamaño de la caja y se conecta con su pedido correspondiente.

En cuanto a las ventas, el diseño incluye las tablas *Cliente* y *Pedido*. La primera guarda la información de contacto y dirección de los compradores, usando el NIF como clave primaria. Asimismo, la tabla *Pedido* registra los datos de la compra (código, precio y fecha) y se relaciona con el cliente mediante una clave ajena. Esta estructura hace posible que cada

lote de la tabla *Caja_llena* se asigne a un pedido concreto, conectando todo el proceso desde que se clasifica un azulejo hasta que llega a su comprador final.

Al mismo tiempo, se ha organizado la entrada de materiales mediante las tablas *Proveedor* y *Caja_Vacia*. Como un proveedor puede suministrar muchos lotes de cajas y un tipo de caja puede comprarse a varios proveedores, se ha creado una tabla intermedia llamada *Compra*. Esta tabla funciona como un registro de las operaciones y utiliza una clave primaria compuesta por el NIF del proveedor, el número de lote y la fecha de compra, guardando también datos económicos como el precio final.

Finalmente, es importante destacar las reglas de integridad que se han aplicado en toda la base de datos. Todas las claves ajenas están configuradas para actualizarse en cascada (ON UPDATE CASCADE). Esto significa que, si se modifica un identificador principal, el cambio se refleja automáticamente en el resto de tablas vinculadas. Por otro lado, se ha bloqueado el borrado en cascada (ON DELETE RESTRICT) para evitar que se eliminen por error registros importantes (como clientes, pedidos o lotes) que ya tienen información productiva asociada. De esta manera, se asegura que no se pierda el histórico de datos de la fábrica. Los scripts con el código SQL para crear todas estas tablas se incluyen en el repositorio del proyecto.

7.4. Repositorio de código en GitHub

En lo que se refiere a la organización de los códigos del proyecto, se ha desarrollado un repositorio en GitHub. El repositorio está estructurado en base a una jerarquía de carpetas separadas por módulos; *Base de Datos*, *Documentación*, *ESP32-S3* y *RoboDK*.

Cada carpeta contiene los códigos desarrollados y un fichero README que explica línea a línea la funcionalidad del código y su correspondiente justificación.

<https://github.com/Pbarter06/Proyecto-Automatizacion>

8. Conclusiones y Recomendaciones

El desarrollo de este proyecto de automatización aplica una solución técnica bajo los avances que implica la Industria 4.0 para el sector cerámico.

Aunque en un principio se planteó la idea de automatizar un proceso agrario, más específicamente, la separación de ganado, se descartó automáticamente debido a los múltiples factores de complicación que suponía esta aplicación tecnológica.

Posteriormente, se buscó un proceso tradicional el cual jugara un gran papel en la Comunidad Valenciana, puesto que como rol de consultora externa nos gustaría impulsar el sector industrial de la zona, siendo éste principalmente el sector de la cerámica. El núcleo del sector de la cerámica se encuentra en la ubicación de Castellón, es por ello por lo que a base de consultar diferentes foros y vídeos sobre fábricas de la zona, se decidió automatizar los procesos que más complicaciones ocasionan tanto a los operarios como a las empresas -haciendo referencia a los procesos que se han automatizado-.

Los puntos claves a destacar del proyecto desarrollado es la alta rentabilidad financiera que supone para la empresa, ya que con una inversión inicial reducida en cuanto a costes generales de la industria -173.297,59€- el proyecto se autofinancia en menos de 2 años. En cuanto a la comunicación entre las unidades robóticas es importante destacar la sincronización IoT mediante el protocolo MQTT y el uso del microcontrolador ESP32-S3. Esta arquitectura descentralizada permite un flujo de mensajes asíncrono y en tiempo real, garantizando que los brazos robóticos (UR5e y UR30) y los sensores de la línea actúen de manera perfectamente coordinada, segura y sin tiempos de espera innecesarios.

Para una futura evolución del proceso automatizado se busca implementar la programación de la Cámara Matrox Iris GTR para conseguir la detección real de taras o roturas del diseño del producto en tiempo real. Así pues, la implementación de una base de datos NoSQL para la gestión de datos masivos como los datos que constantemente envían los sensores ubicados al final de cada cinta mecánica.

Finalmente, la propuesta de automatización para la línea de empaquetado y paletizado de azulejos se decidió implementar mediante robótica colaborativa para dar respuesta directa a la necesidad de eliminar cuellos de botella y mitigar riesgos ergonómicos en la planta de forma flexible y segura. A lo largo de su desarrollo, el proyecto ha comportado desafíos complejos, tales como la integración de sistemas de software y hardware heterogéneos y la programación precisa del gemelo digital, lo que se ha traducido en un valioso aprendizaje práctico sobre la resolución de problemas de ingeniería en entornos industriales reales.

Asimismo, este trabajo ha permitido consolidar y relacionar de forma directa los conocimientos adquiridos en diversas asignaturas de la titulación, tales como Robótica, Redes de Comunicaciones Industriales, Bases de Datos y Gestión de Proyectos. Como recomendaciones finales para el futuro de la planta, se aconseja culminar la integración de la visión artificial para la detección autónoma de defectos y conectar la celda con los sistemas de gestión global ERP/MES, consolidando así una solución plenamente alineada con la Industria 4.0.

9. Referencias

Durante el desarrollo de la memoria se han empleado herramientas de IA orientadas a la optimización de tareas técnicas y como apoyo para la redacción de conceptos complejos.

- **Poblamiento de datos:** se ha utilizado la IA para la generación de registros coherentes y ha facilitado la creación de un entorno de pruebas de datos simulados.
- **Ayuda auxiliar en la realización del código:** durante el desarrollo del proyecto se han utilizado herramientas basadas en inteligencia artificial como apoyo a la programación. Especialmente funciones de autocompletado. Estas herramientas han permitido tanto agilizar el desarrollo, reduciendo así el tiempo dedicado a tareas repetitivas, aunque principalmente su uso se ha basado en la ayuda a la resolución de los errores de código. Es por ello por lo que la herramienta IA principalmente ha sido empleada para la resolución de situaciones bloqueantes a nivel de código en el desarrollo del proyecto.
- **Ayuda auxiliar en la redacción de las justificaciones :** se han utilizado herramientas basadas en la inteligencia artificial como apoyo a la justificación y redacción de apartados. El uso de la IA ha permitido comprender mejor algunos aspectos que resultaban complejos; además de ser una fuente donde consultar y obtener respuestas explicativas sobre aquellos ámbitos donde los materiales y conocimientos impartidos en clase no alcanzaban. Su uso ha permitido reducir el margen de error con respecto a la terminología técnica requerida.

Más específicamente se han empleado las IAs como *Claude* o *Gemini* para resolver principalmente problemas de código. Por otra parte, *Gemini* y *Copilot* han sido herramientas complementarias que han ayudado a la comprensión y redacción de aquellos aspectos más técnicos del proyecto.

10. Anexos

10.1. Manual de Instalación y Funcionamiento

La operatividad y correcto funcionamiento de la célula de automatización virtual dependen estrictamente del orden secuencial en el que se ejecutan sus programas de gestión y de la preparación inicial del entorno. Antes de iniciar cualquier ciclo de producción, es un requisito indispensable asegurar que el sistema se encuentra en un estado limpio, garantizando que todas las variables de control estén inicializadas a cero y que no existan contenedores ni azulejos desposicionados dentro del layout que puedan inducir a errores de colisión.

Esta tarea de puesta a punto previa se simplifica mediante el uso de tres macros nativas desarrolladas directamente en la interfaz gráfica de RoboDK, denominadas "Set a 0", "Set a 0 bd" y "RESET TOTAL". El programa "Set a 0" se encarga de restablecer de forma genérica todas las variables de entorno que el proceso utiliza para la intercomunicación y sincronización de los scripts, incluyendo los registros lógicos que emulan el estado físico de los sensores láser de barrera. De manera complementaria, la macro "Set a 0 bd" realiza una función similar pero acotada exclusivamente a las variables y contadores vinculados con el sistema de almacenamiento de datos. La separación estructural de estos dos programas responde a un criterio de flexibilidad operativa, previendo escenarios donde un programador o técnico requiera reiniciar de forma urgente los componentes de la estación de trabajo pero decida, de manera intencionada, mantener intacto el histórico de registros de la base de datos para no alterar las analíticas de producción acumuladas.

Por último, el procedimiento de limpieza física se completa con la ejecución del programa "RESET TOTAL", el cual comanda el retorno automático de las cinco cintas transportadoras a sus coordenadas de origen y purga de forma masiva cualquier azulejo residual o flotante que haya quedado desubicado fuera de la línea de alimentación principal.

Una vez que la estación de trabajo se encuentra plenamente consolidada en su estado home o inicial, el operador simplemente debe iniciar la secuencia denominada "PROGRAMA FINAL". Esta macro actúa como el cargador principal del entorno, invocando y coordinando en segundo plano la ejecución síncrona de todos los scripts funcionales de Python para que la simulación comience a operar en régimen continuo de forma fluida y exenta de errores lógicos.

Por lo que respecta a la integración y puesta en marcha del hardware físico, la configuración de la unidad embebida ESP32-S3 requiere del entorno de desarrollo Arduino IDE para la compilación y transferencia del firmware. Una vez que el código ha sido cargado exitosamente en el microcontrolador, resulta imprescindible abrir el Monitor Serie del entorno (configurado a una velocidad de 115200 baudios) para realizar el diagnóstico inicial de arranque. En esta consola de depuración, el operador debe verificar visualmente que el dispositivo establece conexión satisfactoria con la red Wi-Fi local y, seguidamente, con el

broker MQTT, confirmando en pantalla el registro de las suscripciones a los distintos canales de mensajería (topics) que gobernarán la celda.

Para garantizar la correcta interacción entre la lógica de red y las señales de campo, es un requisito estricto respetar el esquema de cableado definido en la parametrización del sistema. Dado que las señales de entrada han sido programadas utilizando las resistencias de pull-up internas del microcontrolador (INPUT_PULLUP), los terminales de los pulsadores físicos deben conectarse entre su pin asignado y la referencia de tierra (GND), generando así el estado lógico activo (LOW) al ser accionados. Por su parte, la mayoría de los actuadores visuales operan mediante lógica positiva.

El conexionado exacto de los periféricos de la ESP32-S3 se detalla en la siguiente tabla:

Componente / Función	Tipo de Señal	Pin (GPIO)	Lógica de Activación
Botón: Azulejo Bueno	Entrada (Pull-up)	11	LOW (Pulsado a GND)
Botón: Azulejo Malo	Entrada (Pull-up)	12	LOW (Pulsado a GND)
Botón: Azulejo Defectuoso	Entrada (Pull-up)	13	LOW (Pulsado a GND)
Botón: Vaciar Palet 1	Entrada (Pull-up)	9	LOW (Pulsado a GND)
Botón: Vaciar Palet 2	Entrada (Pull-up)	10	LOW (Pulsado a GND)
LED: Estado Palet 1 (Lleno)	Salida	17	HIGH (Nivel Alto)
LED: Estado Palet 2 (Lleno)	Salida	18	HIGH (Nivel Alto)
LED: Funcionamiento del Sistema	Salida (Integrado)	2	LOW (Lógica Invertida)

10.2. Manual del Programador

El desarrollo del software para esta célula automatizada se ha diseñado bajo el paradigma de **programación modular y distribuida**, utilizando el lenguaje Python como eje integrador debido a su flexibilidad para interactuar simultáneamente con APIs de simulación, bases de datos y protocolos de comunicación industrial. La filosofía de diseño se ha centrado en la creación de hilos de ejecución independientes (bucles infinitos controlados) para cada entidad física del entorno (robots y cintas transportadoras). Esta independencia evita el acoplamiento de código y asegura que un retraso en una estación no bloquee por completo el procesamiento general del sistema, emulando fielmente el comportamiento de una red de PLCs o controladores industriales en una planta real.

Para mitigar los problemas clásicos de la programación síncrona en entornos virtuales (como la saturación del procesador por hilos en espera (*polling*)) se han implementado subrutinas de retardo controlado mediante la librería *time*. Asimismo, el almacenamiento de datos se ha delegado por completo a un sistema gestor de bases de datos relacionales (PostgreSQL) a través de la librería *psycopg*, abstrayendo la lógica de control del almacenamiento físico de la producción y garantizando los principios de la Industria 4.0.

La arquitectura del software en RoboDK se segmenta en dos grandes bloques operativos: las macros nativas de entorno y el ecosistema de scripts en Python. Las macros nativas se ejecutan directamente sobre la interfaz gráfica de RoboDK y están destinadas a la gestión y preparación del entorno virtual, destacando "PROGRAMA FINAL" como el secuenciador encargado de lanzar todos los subprocesos de código de forma ordenada, "RESET TOTAL" para la restauración física de las cintas transportadoras y la purga de elementos flotantes, y "SET A 0" para la inicialización a valores nulos de las variables globales del sistema.

Por su parte, el ecosistema de scripts de Python se divide estructuralmente en tres módulos funcionales independientes y coordinados:

- **Módulo de Transporte y Flujo de Materiales:** Compuesto por los scripts individuales que gobiernan las cinco cintas transportadoras. Su estructura lógica interna define en primera instancia las variables de entorno de RoboDK y los marcos de referencia, para posteriormente ejecutar un bucle asíncrono que desplaza los azulejos hasta que la función nativa de colisión detecta el flanco de subida de las fotocélulas SICK virtuales. Adicionalmente, las subrutinas de las cintas 1 y 2 actúan como pasarelas de datos, encargándose de actualizar la base de datos al llenarse una caja antes de simular el empaquetado bajo el arco de la célula.
- **Módulo de Comunicación y Eventos MQTT:** Encargado de la mensajería bidireccional entre la simulación y el hardware embebido (ESP32-S3). Este bloque se subdivide jerárquicamente en scripts de transmisión y scripts de recepción. Los scripts de recepción (*LeerMQTTS* y *LeerMQTTB*) monitorizan de forma ininterrumpida el *broker* MQTT para capturar las pulsaciones del panel físico. Al recibir un evento, lo

delegan a los controladores lógicos *RobotControllerS* —que actualiza la variable global *ColaAzulejos* emulando las señales de defecto de la cámara — y *RobotControllerB*, encargado de procesar órdenes manuales de mantenimiento como la evacuación de palets completos. En contraposición, los scripts de transmisión se ejecutan por flanco ante eventos específicos para publicar mensajes de control (ON/OFF) destinados a conmutar los LEDs de estado del panel físico.

- **Módulo de Control de Estaciones Robóticas:** Alberga los algoritmos principales de decisión cinemática para los dos brazos industriales. El script del robot UR5e (clasificación y apilado) implementa una lógica de selección basada en una cola de datos globalizada, donde tras comprobar la coincidencia de presencia física en el sensor y datos en cola, realiza una inserción SQL de trazabilidad mediante un cursor activo. Posteriormente, mediante una estructura de control condicional, discrimina las piezas rotas desviándolas cinemáticamente al descarte, o ejecuta una rutina de sujeción por vacío (*setParentStatic*) y colocación matricial dinámica para las piezas aptas o defectuosas. Finalmente, el script del robot UR30 (paletizado) opera bajo un esquema de prioridades que se activa ante los sensores de cajas llenas, ejecutando transformaciones algebraicas tridimensionales para calcular el mosaico exacto de paletizado en el palet correspondiente y disparando programas paralelos de alerta cuando las estaciones alcanzan su capacidad máxima.

Complementando la arquitectura de simulación, el firmware del sistema embebido (ESP32-S3) se ha desarrollado en C/C++ (entorno Arduino) bajo un paradigma orientado a eventos y puramente asíncrono. Para facilitar su mantenimiento, escalabilidad y comprensión por parte de futuros desarrolladores, el código fuente se ha segmentado en los siguientes módulos funcionales:

1. Módulo de Configuración y Diagnóstico

- **config.h:** Archivo central de parametrización. Agrupa las credenciales de red, el mapeo de pines físicos (GPIO) y las definiciones estáticas de los topics MQTT.
- **logger.ino:** Implementa un sistema de trazas por consola serie clasificado por niveles de severidad (TRACE, INFO, ERROR, etc.), permitiendo ajustar la verbosidad de la depuración desde una única macro global.

2. Módulo de Conectividad y Pasarela MQTT

- **wifi_lib.ino:** Encapsula la conexión inalámbrica y sus rutinas de auto-recuperación para evitar bloqueos ante caídas de red.
- **mqt_lib.ino:** Administra la comunicación IoT mediante la librería PubSubClient. Contiene la función callback, punto de entrada donde se ejecuta el algoritmo de deduplicación (DJB2 y búfer circular) para filtrar ráfagas de mensajes repetidos antes de derivarlos a la lógica central.

3. Módulo de Interacción de Hardware (I/O)

- **setup.ino**: Ejecuta la rutina de inicialización, configurando los modos de los pines y estableciendo los estados eléctricos seguros de arranque.
- **buttons.h** y **comunicaciones_buttons.ino**: Definen la abstracción de datos BotonMQTT y ejecutan el escaneo iterativo de los pulsadores. Emplean temporizadores basados en *millis()* para aplicar un control antirrebote (debounce) no bloqueante, publicando eventos en la red al detectar flancos de bajada.
- **comunicaciones_led.ino** y **funciones.ino**: Actúan como handlers de salida. Procesan los comandos textuales recibidos desde la aplicación central vía MQTT y los traducen en señales eléctricas para conmutar los indicadores luminosos de estado.

Finalmente, la integración de estos módulos se orquesta mediante un *loop()* principal que cede tiempo de procesador iterativamente al mantenimiento de la red (*mqttClient.loop()*) y a la lectura de sensores. Esta arquitectura evita el uso de bloqueos o pausas (delays), garantizando una latencia mínima en la sincronización bidireccional con el gemelo digital.

11. Bibliografía

OnRobot. (s. f.). *VGP20 Vacuum Gripper: Datasheet*.

https://onrobot.com/storage/datasheets/datasheet_vgp20.pdf

SMC Corporation. (s. f.). *Air Gripper for Collaborative Robots: Series ZXP7*.

https://content2.smcetech.com/pdf/ZXP7_01_X1.pdf

Universal Robots. (s. f.). *UR5e e-Series: Datasheet*.

https://www.universal-robots.com/media/1807465/ur5e_e-series_datasheets_web.pdf

Universal Robots. (s. f.). *UR30: Datasheet*.

https://www.universal-robots.com/media/1829274/ur30_datasheet_web.pdf

European Pallet Association e.V. (s.f.). *Load carriers overview*. EPAL.

<https://www.epal-pallets.org/eu-en/load-carriers/overview>

Adrihosan. (s.f.). *Tamaños de azulejos: Guía completa*.

https://www.adrihosan.com/tamanos-de-azulejos/?srsltid=AfmBOoqd1XjYDc9faxlR_Dg7ZeVY-Z-1DQKECylMuN2YmCTk53IiraE5

RoboDK Inc. (s.f.). *SICK WL4S Laser Sensor*. Robo DK Library.

<https://robodk.com/object/es/SICK-WL4S-Laser-Sensor>

SICK AG. (s.f.). *WSL4S-3V2232 | W4 | Fotocélulas*.

<https://www.sick.com/cl/es/catalog/productos/sensores-de-deteccion/fotocelulas/w4/wl4s-3v2232/p/p222149>

SICK AG. (s.f.). *WL4S-3P2232: Data sheet* [Archivo PDF].

https://www.sick.com/media/pdf/8/48/148/dataSheet_WL4S-3P2232_1042078_en.pdf

Ceramic Connection. (s.f.). *Azulejo Pure Negro Brillo 20x20*.

<https://ceramicconnection.com/es/azulejo/pure-negro-brillo-20x20.html>

Matrox Imaging. (s.f.). *Matrox Iris GTR: Data sheet* [Archivo PDF].

https://www.nctechsales.com/uploads/datasheet/file/280/iris_gtr.pdf

RoboDK Inc. (s.f.). *Matrox Iris GTR Camera*. RoboDK Library.

<https://robodk.com/object/es/Matrox-Iris-GTR-Camera>

Visión Artificial. (s.f.). *Matrox Iris GTR con MIL*.

<https://www.vision-artificial.es/producto/camaras-inteligentes/matrox-iris-gtr-con-mil/#tab-description>

El Periódico del Azulejo. (2024, 21 de mayo). *Polytec revoluciona la logística cerámica con un robot de picking que automatiza la preparación de palés multirreferencia*.

<https://www.elperiodicodelazulejo.es/industria/polytec-revoluciona-la-logistica-ceramica-con-un-robot-de-picking-que-automatiza-la-preparacion-de-pales-multirreferencia-LL2148184>

Gruppo Tecnoferrari S.p.A. (s.f.). *La evolución de la automatización en el sector cerámico.*

<https://www.tecnoferrari.it/es/post/la-evoluci%C3%B3n-de-la-automatizaci%C3%B3n-en-el-sector-cer%C3%A1mico>

Robottions. (s.f.). *Paletizador cerámico.*

<https://www.robottions.com/paletizador-ceramico/>

System Ceramics S.p.A. (s.f.). *Tecnologías para la industria cerámica.*

<https://www.systemceramics.com/es>

Asociación Técnica Argentina de Cerámica. (2006, Abril). Normas técnicas. *Cerámica y cristal*, (138).

<https://www.ceramicaycristal.com/cc138pdf/normas.pdf>

Asana. (2025, 18 de febrero). *Prueba de concepto (POC): qué es y cómo implementarla.*

<https://asana.com/es/resources/proof-of-concept>

Asociación para el Progreso de la Dirección. (2023, 24 de abril). *Tipos de KPIs: ejemplos para cada departamento.* APD.

<https://www.apd.es/tipos-de-kpis/>

Universal Robots. (s.f.). *ISO/TS 15066: Conozca todo sobre la especificación ISO para robots colaborativos.* Universal Robots Blog.

<https://www.universal-robots.com/mx/blog/isots-15066-conozca-todo-sobre-la-especificacion-iso-para-robots-colaborativos/>

Asociación Española de Normalización. (2019). *Seguridad de las máquinas. Equipo eléctrico de las máquinas. Parte 1: Requisitos generales* (Norma UNE-EN 60204-1:2019).

<https://www.tuvsud.com/es-es/centro-recursos/articulos-de-opinion/requisitos-une-en-602041-equipos-electricos-maquinas>

Directiva 2006/42/CE del Parlamento Europeo y del Consejo, de 17 de mayo de 2006, relativa a las máquinas y por la que se modifica la Directiva 95/16/CE. (2006). *Diario Oficial de la Unión Europea*, L 157.

<https://www.boe.es/doue/2006/157/L00024-00086.pdf>

Directiva 2014/30/UE del Parlamento Europeo y del Consejo, de 26 de febrero de 2014, sobre la armonización de las legislaciones de los Estados miembros en materia de compatibilidad electromagnética. (2014). *Diario Oficial de la Unión Europea*, L 96.

<https://eur-lex.europa.eu/ES/legal-content/summary/reducing-interference-between-electrical-and-electronic-devices.html>

International Organization for Standardization. (2016). *Robots and robotic devices — Collaborative robots* (Norma ISO/TS 15066:2016).

<https://cdn.standards.iteh.ai/samples/62996/37daa360128440ccbc94e9782c9e1748/ISO-TS-15066-2016.pdf>